

Scalable Vertiport Hub Location Selection for Air Taxi Operations in a Metropolitan Region

Liting Chen,^a Sebastian Wandelt,^a Weibin Dai,^a Xiaoqian Sun^{a,*}

^aSchool of Electronic and Information Engineering, Beihang University, 100191 Beijing, China

*Corresponding author

Contact: 98chenliting@gmail.com (LC); wandelt@informatik.hu-berlin.de  <https://orcid.org/0000-0003-0088-9287> (SW); daiweibin@buaa.edu.cn (WD); qqian.sun@gmail.com,  <https://orcid.org/0000-0002-8713-142X> (XS)

Received: February 24, 2020

Revised: October 20, 2020; March 21, 2021

Accepted: June 15, 2021

Published Online in Articles in Advance:
November 29, 2021

<https://doi.org/10.1287/ijoc.2021.1109>

Copyright: © 2021 INFORMS

Abstract. On-demand air mobility services, often called air taxis, are on the way to revolutionize our urban/regional transportation sector by lifting transportation to the third dimension and thus possibly contribute to solving the congestion-induced transportation deadlock many metropolitan regions face today. Although existing research mainly focuses on the design of efficient vehicles and specifically battery technology, in the near future, a new question will arise: Where to locate the vertiports/landing pads for such air taxis? In this study, we propose a vertiport location selection problem. In contrast to existing studies, we allow the demand to be distributed over the whole metropolitan area, modeled as a grid, and exclude certain grid cells from becoming hubs, for example, because of safety/geographical constraints. The combination of these two contributions makes the problem intriguingly difficult to solve with standard solution techniques. We propose a novel variable neighborhood search heuristic, which is able to solve 12×12 grid instances within a few seconds of computation time and zero gaps in our experiments, whereas CPLEX needs up to 10 hours. We believe that our study contributes toward the scalable selection of vertiport locations for air taxis.

Summary of Contribution: The increasing interest in opening the third dimension, that is, altitude, to transportation inside metropolitan regions raises new research challenges. Existing research mainly focuses on the design of efficient vehicles and control problems. In the near future, however, the actual operation of air taxis will lead to new set of operations research problems for so-called air taxi operations. Our contribution focuses on the optimization of vertiports for air taxi operations in a metropolitan region. We choose to model the problem over a grid-like demand structure, with a novel side constraint: selected grid cells are unavailable as hubs, for example, because of environmental, technical, cultural, or other reasons. This makes our model a special case in between the two traditional models: discrete location and continuous location. Our model is inherently difficult to solve for exact methods; for instance, solving a grid of 12×12 grid cells needs more than 10 hours with CPLEX, when modeled as a discrete location problem. We show that a straightforward application of existing neighborhood search heuristics is not suitable to solve this problem well. Therefore, we design an own variant of mixed variable neighborhood search, which consists of novel local search steps, tailored toward our grid structure. Our evaluation shows that by using our novel heuristic, almost all instances can be solved toward optimality.

History: Accepted by Erwin Pesch, Area Editor for Heuristic Search & Approximation Algorithms.

Funding: This study was funded by the National Natural Science Foundation of China [Grants 61851110763, 61861136005, and 71731001].

Supplemental Material: The online appendix is available at <https://doi.org/10.1287/ijoc.2021.1109>.

Keywords: air taxi • vertiport selection problem • variable neighborhood search

1. Introduction

The vision of opening the third dimension, that is, altitude, for urban transportation has gained substantial interest recently, with significant engineering efforts to explore the so-called *on-demand air mobility* (ODAM). ODA, as a novel transportation concept, has the potential to radically change our understanding of mobility (Holmes et al. 2004, Schippl et al. 2013, Holmes 2016). Industry and academia are spending a

large number of resources on the development of ODA. The National Aeronautics and Space Administration (NASA) advertises ODA within a strategic framework (Holmes et al. 2017). Uber released a white paper describing their views regarding on-demand aviation (Holden and Goel 2016). Besides NASA and Uber, many other organizations are showing active interest in launching air taxi programs. These efforts highlight the near-future widespread adoption potential of

air taxi services. Holden and Goel 2016 provide a detailed analysis of air taxi market feasibility across four dimensions: vehicle, infrastructure and operations, rider experience, and economics. Several studies have focused on the technical feasibility of ODAM, including vehicle design (Stumpf et al. 2018), technology for electronic drones (Polaczyk et al. 2019), and distributed electronic propulsion technology (Kim et al. 2018). However, there are still several challenges to overcome before the real implementation of ODAM. Many air taxi concepts follow the idea of taking-off and landing vertically (called VTOL for vertical take-off and landing). Accordingly, one of the greatest operational barriers is to select locations for vertiports. Optimized locations of these vertiports according to demand distribution are vital in order to minimize the total travel cost and travel time (Sun et al. 2018). Constraints on land availability/use, airspace restrictions, and other geographical properties limit the locations of vertiports (Holden and Goel 2016, Robinson et al. 2018). Current state-of-the-art studies mostly focused on technical feasibility. However, limited research has focused on infrastructure planning for air taxis.

In this study, we select vertiport locations in a given metropolitan area with the aim of minimizing the total travel cost. The problem of our study has its roots in hub location problems. Since the seminal work by O’Kelly (1986), hub location problems have been studied for over three decades. Based on the distribution of travel demands, hub location problems are mainly divided into two groups: (1) discrete hub location problems (Campbell 1994) with travel demands among discrete nodes and (2) continuous hub location problems (Campbell 1993) with travel demands on continuous regions. We make two assumptions, which lead to a distinct novel problem. First, following the idea of a grid cell-based framework (Sun et al. 2018), we allow the demand to be distributed over the whole metropolitan area, modeled as a grid. In this way, we have a fine-grained representation of transportation demands. Dividing the city into n grid cells, we use the center of each grid cell to represent itself and set its travel demands as the sum of the demand inside its domain. Accordingly, an uncapacitated single allocation p -hub median model is designed. Because of the adjacency property of grid cells, this model is different from the discrete hub location problem based on nodes. Second, in the real world, some regions cannot be used as hubs because of safety or environmental reasons. Therefore, some grid cells are being declared as nonhubable, that is, not allowed to become hubs in our model. As far as we know, this constraint about nonhubable regions was not been explored in the literature yet. In general, grid-based model can be seen as an approximation for the continuous model. Solution techniques for discrete hub location

problems do not scale up well with the size of the model; solution techniques for continuous hub location problems cannot handle the constraints on nonhubable areas well. Accordingly, we propose a highly efficient solution heuristic based on variable neighborhood search (VNS). Our heuristic is not a straightforward extension of existing VNS algorithms but rather contributes several new local search steps, which greatly increase the overall speed and provide guidance for informed local search. We also develop a new VNS for the classic uncapacitated single allocation p -hub median problem. The contributions of our paper are summarized as follows:

1. We model the problem of vertiport hub selection, with the goal of locating exactly p vertiports while minimizing the total travel cost and obeying constraints on nonhubable areas. We show that the model is hard to solve for standard solvers. Moreover, we implemented a standard variable neighborhood search heuristic and show that it does not scale up well with larger grids either.

2. We propose a novel mixed variable neighborhood descent as the step of local searches that includes Alternate, Reassign, Explore, Substitute, and SwapK. Although Alternate and Reassign were previously studied, Explore, SwapK, and Substitute are three new and effective local searches specifically proposed by us. Based on a set of experiments, it is shown that the proposed algorithm outperforms other methods or techniques in terms of runtime and solution quality.

3. A new general variable neighborhood search is proposed for the classic uncapacitated single allocation problem. Combined with two new local searches and efficient computation of the difference of the objective function, the new general variable neighborhood search generates all the best-known results for the instances we tested in a reasonable time.

The remainder of this paper is organized as follows. We provide a review of related literature in Section 2. The novel grid-based model is proposed in Section 3. Section 4 presents our VNS-based heuristic algorithm. Section 5 reports the experimental results and Section 6 concludes the paper.

2. Literature Review

In this section, we first provide a review of the literature about the vertiport selection problem and identify current research gaps in Section 2.1. Given that our problem is closely related to the hub location problem, we review literature about HLP (hub location problem) in Section 2.2.

2.1. Vertiport Location Problem

Although some researchers studied the requirements and constraints of the locations of vertiports, limited research has focused on the infrastructure planning for air taxis. Lim and Hwang (2019) used the K-means

algorithm to cluster the commuting data and select vertiports for Seoul, the capital city of South Korea. However, they analyzed the three highest-demand routes instead of the traffic in the whole metropolitan area in Seoul. While focusing on these three routes, the authors launch vertiports to alleviate traffic aggregation. Besides, they did not set up a general optimization framework for this problem. Rath and Chow (2019) formulated the vertiport selection problem as an extension of HLP with the case study of New York City. However, they focused on special destinations and only analyzed trips with airports as destinations in their case study. About the constraints and requirement of vertiports, Lim and Hwang (2019) only set the vertiports at existing airports and cloverleaves, which are potential locations of vertiports proposed by NASA (Antcliff et al. 2016). Rath and Chow (2019) did not take this factor into account.

2.2. Hub Location Problem

HLP is an extension of the facility location problem and has attracted active interest since O’Kelly (1987) introduced the first mathematical formulation. Hubs are facilities that collect and transfer flow between origin-destination pairs. The goal of the problem is to find the optimal hub locations and the allocation of nonhub nodes. In the hub location problem, hubs are usually fully interconnected (complete network), transfers between hubs are reduced by a discount factor α , and direct links between spoke nodes are not allowed (Farahani et al. 2013).

According to Alumur and Kara (2008), hub location problems can be classified based on the allocation types, complete (or incomplete) network, or capacitated (or uncapacitated) hubs. For the design of the access network, most studies on HLP assume single allocation (Chen 2007, Elhedhli and Wu 2010, Contreras et al. 2011) or multiple allocation (de Camargo et al. 2008, Köksalan and Soylu 2010). Different design characteristics of the access network are applied in different applications. Using single allocation, the structure of the access network is simple, and the number of access links is minimized. In some cases, single and multiple allocation may be two extremes. Different from single allocation and multiple allocation, Yaman (2011) proposed the r -allocation strategy, where each node can be assigned to r hubs. The r -allocation problem is a generalization of the single and multiple allocation problem. For the design of the interhub links, most literature focused on the complete networks (Peiró et al. 2014, Martí et al. 2015, Hoff et al. 2017). However, some real-life networks from various applications do not necessarily operate fully connected hub networks. Some literature contributes to the design of incomplete networks recently (de Camargo et al. 2008, Köksalan and Soylu 2010). Alumur et al. (2009) proposed models on four fundamental hub location problems with an incomplete but connected interhub network. They used new

constraints on the number of hub links to design an incomplete network. Some research used the setup cost of hub links to limit the number of hub links (Contreras et al. 2010, O’Kelly et al. 2015). Additionally, some other hub location problems are introduced recently, such as competitive hub location problem (Lüer-Villagra and Marianov 2013), hub location problem with stochastic elements (Correia et al. 2018), intermodal hub location problem (He et al. 2015), problems with reliability consideration (Mohammadi et al. 2019), and planar hub location problem (Damgacioglu et al. 2015).

Given that it is intrinsically difficult to solve large-scale hub location problems using exact techniques, many heuristic algorithms have been proposed for different variants of hub location problems. Abdinnour-Helm (1998) proposed a hybrid heuristic based on genetic algorithm and tabu search and solved the CAB (Civil Aeronautics Board) data set efficiently. Cunha and Silva (2007) proposed a hybrid algorithm for the single allocation hub location problem. They used a genetic algorithm to determine the hub set and simulated annealing to allocate the spoke nodes. Corberán et al. (2016) studied the capacitated hub location problem with modular links and proposed a solving method based on strategic oscillation. de Sá et al. (2015) studied hub networks with multiple lines. In their model, the hubs are located in some sets of lines. They proposed neighborhood search, greedy randomized adaptive search, and adaptive large neighborhood search to solve this model.

In this paper, we studied the uncapacitated single allocation p -hub median problem (USApHMP). O’Kelly (1987) formulated the first discrete USApHMP model as a quadratic integer problem. In the model of USApHMP, there are n nodes in a many-to-many distribution network. Every nonhub node is allocated to a single hub, whereas each hub node is assigned to itself. Direct links between two nonhub nodes are forbidden. The basic framework is to select p hubs and minimize the total travel cost. Campbell (1994) proposed the corresponding linear programming formulation with $O(n^4)$ variables. A new linear programming formulation was proposed by Ernst and Krishnamoorthy (1996) with $O(n^3)$ variables. Our grid-based model is a modification of this model. In the model proposed by Ernst and Krishnamoorthy (1996), the hubs are allowed to be selected from the whole network, whereas in our model, hubs are not allowed to be selected from the nonhubable grid cells.

USApHMP is known to be an NP-hard (non-deterministic polynomial-time hard) problem (Alumur and Kara 2008). Various solution algorithms have been proposed to solve problems with large size (Sun et al. 2017). These approaches include exact techniques like Benders decomposition (de Camargo et al. 2008) and quadratic programming, and heuristics like simulated annealing (Ernst and Krishnamoorthy 1999), branch and bound (Rodríguez-Martín et al. 2014), tabu search (Calik

et al. 2009), genetic algorithm (Kratica et al. 2011), and variable neighborhood search (Ilić et al. 2010). Variable neighborhood search has been commonly applied to solve large-scale problems in recent years. In the variable neighborhood search, an initial solution is followed by a sequence of local searches that are used to obtain local optima. A perturbation phase is applied to get out of the local minima. Ilić et al. (2010) proposed a general variable neighborhood search to solve the USApHMP. Their heuristic outperforms the existing ones in terms of solution quality and computing time. They improved the best-known solutions for some instances and were the first to present results for problems with up to 1,000 nodes. VNS has also been applied to solve more complicated HLP such as the uncapacitated r -allocation p -hub median problem (Martí et al. 2015, Todosijević et al. 2017), the single allocation model of the capacitated intermodal hub network design problem proposed by Serper and Alumur (2016), and incomplete hub location problems (Dai et al. 2019). One of the problems in our study, the irregular distribution of demand over a region, could be addressed by demand point aggregation, which has been studied in the past (Francis et al. 2004, Emir-Farinas and Francis 2005). Although aggregation decreases the cost of solving the problem, it significantly increases the error of the model. To date, it is still an open question of how to make a tradeoff between the benefits and the model error (Francis et al. 2009).

3. Model Formulation

In our model, we assume that in every grid cell, there are already vertistops established for aircraft taking off or landing and the model aims at locating p vertiports to reduce the overall cost. According to Holden and Goel (2016), vertistops are single VTOL pads with minimal infrastructure (costs), whereas vertiports are VTOL hubs with multiple takeoffs and landing pads, as well as recharging infrastructure. Given the travel demands of the whole city and the hubable area, our model aims at minimizing the transportation cost by locating p hub vertiports and designing the hub network. A passenger will take a no-more-than-three-leg trip of air taxi, via single hub or two hubs, from its origin grid cell to the destination grid cell. The travel demand is distributed over the whole metropolitan area, and we model the whole area as $n_b \times n_b$ grid cells. All the demand inside the grid cell is aggregated in its center. The coordinate (i_x, i_y) , $(0 \leq i_x < n_b, 0 \leq i_y < n_b)$ represents each grid cell in the metropolitan area G .

The amount of flow between grid cell (i_x, i_y) and grid cell (j_x, j_y) is denoted as $w_{i_x i_y j_x j_y}$, and the distance between these two cells is denoted as $d_{i_x i_y j_x j_y}$. We use set G_N to represent the set of grid cells with demand. Empty grid cell with no demand, for example, water area, is not included in G_N . G_H is used to denote the set of hubable grid cells. The number of nodes in G_N is denoted as n , whereas the number of nodes in G_H denoted as n_h .

Given that single allocation can generate the simplest network of access links and ease the air traffic management when first launching air taxi service, we use single allocation cases. When using single allocation strategy, given that one grid cell is only connected to one hub, there will be few intersections between any access links. Thus, potential collision of air taxis in the access links could be avoided. Because larger air taxis can be used in the hub links between vertiports, the travel cost between vertiports is reduced by a discount factor α . Additionally, we use χ and δ to represent the cost of unit distance for collection (origin-hub) links and distribution (hub-destination) links. The terms $O_{i_x i_y}$ and $D_{i_x i_y}$ denote the amount of flow from and to (i_x, i_y) , respectively. Based on the USApHMP model provided by Ernst and Krishnamoorthy (1996), we propose a model for vertiport location problem. Based on the decision variables (Table 1), the hub location problem with non-hubable area is modeled as follows:

$$\min \sum_{(i_x, i_y) \in G_N} \sum_{(k_x, k_y) \in G_H} \chi d_{i_x i_y k_x k_y} Y_{i_x i_y k_x k_y} O_{i_x i_y}$$

$$+ \sum_{(i_x, i_y) \in G_N} \sum_{(k_x, k_y) \in G_H} \delta d_{k_x k_y i_x i_y} Y_{i_x i_y k_x k_y} D_{i_x i_y} \\ + \sum_{(i_x, i_y) \in G_N} \sum_{(k_x, k_y) \in G_H} \sum_{(m_x, m_y) \in G_H} \alpha d_{k_x k_y m_x m_y} X_{k_x k_y m_x m_y}^{i_x i_y}, \quad (1)$$

$$s.t. \sum_{(k_x, k_y) \in G_H} Z_{k_x k_y} = p, \quad (2)$$

$$\sum_{(k_x, k_y) \in G_H} Y_{i_x i_y k_x k_y} = 1, \quad \forall (i_x, i_y) \in G_N, \quad (3)$$

$$Y_{i_x i_y k_x k_y} \leq Z_{k_x k_y}, \quad \forall (i_x, i_y) \in G_N, \quad \forall (k_x, k_y) \in G_H, \quad (4)$$

$$\sum_{(m_x, m_y) \in G_H \setminus \{(k_x, k_y)\}} X_{k_x k_y m_x m_y}^{i_x i_y} + \sum_{(j_x, j_y) \in G_N} w_{i_x i_y j_x j_y} Y_{j_x j_y k_x k_y} \\ = O_{i_x i_y} Y_{i_x i_y k_x k_y} + \sum_{(m_x, m_y) \in G_H \setminus \{(k_x, k_y)\}} X_{m_x m_y k_x k_y}^{i_x i_y},$$

$$\forall (i_x, i_y) \in G_N, \quad \forall (k_x, k_y) \in G_H, \quad (5)$$

$$Y_{i_x i_y k_x k_y} \in \{0, 1\}, \quad \forall (i_x, i_y) \in G_N, \quad \forall (k_x, k_y) \in G_H, \quad (6)$$

$$X_{k_x k_y m_x m_y}^{i_x i_y} \geq 0, \quad \forall (k_x, k_y), (m_x, m_y) \in G_H, \quad \forall (i_x, i_y) \in G_N. \quad (7)$$

Table 1. Decision Variables for the Grid-Based Hub Location Problem

Variable	Description
$X_{k_x k_y m_x m_y}^{i_x i_y}$	Flows routed on hub link starting from (k_x, k_y) to (m_x, m_y) originated from grid (i_x, i_y) , $((i_x, i_y) \in G_N, (k_x, k_y), (m_x, m_y) \in G_H)$
$Y_{i_x i_y k_x k_y}$	1 if grid cell (i_x, i_y) is allocated to hub (k_x, k_y) , 0 otherwise, $((i_x, i_y) \in G_N, (k_x, k_y) \in G_H)$
$Z_{k_x k_y}$	1 if a hub is located in grid cell k , 0 otherwise, $((k_x, k_y) \in G_H)$

The objective function (1) is to find the most economical allocation for all grid cells. Constraint (2) guarantees that the number of hubs is a given number p , and only grid cells in G_H are selected as hubs. Constraint (3) forces each grid cell to be allocated to only one open hub in G_H . Constraint (4) ensures that each grid cell will only be allocated to hub grid cells. Constraint (5) is the flow equivalent equation for every intermediate hub grid cell. The left-hand side is the amount of flow needed to be transported from (i_x, i_y) via hub (k_x, k_y) to other nodes, including hub nodes (the first term) and spoke nodes directly connected to (k_x, k_y) (the second term). The right-hand side represents the flow transported from (i_x, i_y) to hub (k_x, k_y) , including (k_x, k_y) as the first hub (the first term) or second hub (the second term). Constraint (5) guarantees the flow needed to be transported from (i_x, i_y) via hub (k_x, k_y) to other nodes equals to the flow that is transported from (i_x, i_y) to hub (k_x, k_y) . Finally, Constraints (6) and (7) are the variable constraints of this model.

Different from the previous model proposed by Ernst and Krishnamoorthy (1996), we use the coordinates of each grid cell for representation. The spatial correlation between grid cells can be obtained using the index. For example, the neighbor grid cell of (i_x, i_y) can be accessed via $(i_x \pm 1, i_y \pm 1)$. This model involves $(nm_h^2 + nm_h + n_h)$ decision variables and requires $(1 + n + 2nm_h)$ linear constraints, whereas the model of Ernst and Krishnamoorthy (1996) contains $O(n^3)$ variables and constraints. Although this model does not provide as tight as linear bound as the four-index problem proposed by Campbell (1994), the reduced number of variables and constraints make it possible to solve large instances. Given the nonhubable area, the search space of this model is smaller than the original model. However, the number of variables and constraints still grows exponentially when n increase, thus making the model difficult to be solved.

In this model, we do not allow direct connection, given that it will make the management of air space difficult. According to Holden and Goel (2016), VTOLs are unlikely to carry passengers directly door-to-door but instead between vertiports and vertistops. We assume that the hubs and hub links have enough capacity to handle all of the flows; thus, the results of the flow of every hub and hub link can be used to guide the future setup of air taxi infrastructure. The capacity of vertiports is difficult to model given it will be affected by several factors such as the frequency and scheduling of air taxis. Because we do not take the time dimension into account, we do not model it in our problem. When the capacity of each hub is available, constraints on capacity can be added to the model and generate a capacitated vertiport hub location problem. The number of hubs is fixed in our model. However, this can be relaxed when the initial cost of hubs is known. According to Holden and Goel (2016),

the initial cost of vertiport infrastructure is mainly composed of the building modification (such as retrofit of garage structure) and voltage recharger, but the relationship between the reduced travel cost and the initial cost is hard to quantify; thus, we do not model it in the current study. Given the cost of building a vertiport, we can range p from one to a given number to get solutions of different p and compare the results of different p to decide the number of hubs to be established.

4. VNS-Based Heuristic

In this section, our novel VNS-based heuristic for the grid-based problem is introduced. First, the representation of the solution is described, followed by a description of the algorithm, including the initialization process, local search, shaking step, and the variable neighborhood descent. Last, we introduce a new VNS for classic USApHMP.

4.1. Representation

We use the assignment array A of length n to represent the assignment of every grid cell in G . H is used to denote the current hub set of the assignment. Index i ($i = i_x n_b + i_y$) is used to represent each grid cell. In order to simplify the computation of the new overall cost, two matrices W_{in} and W_{out} are defined in Equations (8) and (9). The term $W_{in}[i][k]$ denotes the flow routed from hub k on $A[i]$ to grid cell i , and $W_{out}[i][k]$ denotes the flow routed on $A[i]$ to hub k from grid cell i . We use W_{in}^n and W_{out}^n to represent the updated W_{in} and W_{out} , respectively, when new assignment is accepted. We also introduce $W_H[k][m]$ to indicate the flow from hub k to hub m .

$$W_{in}[i][k] = \sum_{j \in \{j|A[j]=k\}} w_{ji} \quad (8)$$

$$W_{out}[i][k] = \sum_{j \in \{j|A[j]=k\}} w_{ij} \quad (9)$$

$$\begin{aligned} W_H[k][m] &= \sum_{i \in \{i|A[i]=k\}} \sum_{j \in \{j|A[j]=m\}} w_{ij} = \sum_{i \in \{i|A[i]=k\}} W_{out}[i][m] \\ &= \sum_{j \in \{j|A[j]=m\}} W_{in}[j][k] \end{aligned} \quad (10)$$

4.2. Initialization

During initialization, we first select p grid cells with the largest demand in G_H as hubs and then assign all grid cells to their closest hub.

4.3. Local Searches

We use five local searches including Alternate, Reassign, Explore, Substitute, and SwapK. Alternate and Reassign are two frequently used local search methods when applying VNS to solve HLP (Ilić et al. 2010). However, we simplify the process to calculate the difference of Alternate. Our calculation is around $\frac{p}{2}$ times faster than that one introduced in (Ilić et al. 2010). In addition to Alternate and

Reassign, we propose three novel neighborhood search methods including Explore, Substitute, and SwapK.

4.3.1. Alternate. We use a cluster to represent the set of grid cells (nodes) with the same hub. In the first neighborhood of local search, all clusters are preserved and we only change the location of one hub in one cluster. With the representation of assignment array A , all clusters are generated after accessing the elements in A once. In Cluster $\mathcal{C}$ with m_h grid cells and hub h , the set of candidate hubs should contain all the grid cells in $\mathcal{C}$ and also in G_H except for h . We denote the set of candidate hub set as S_{cand} ($S_{cand} = \mathcal{C} \cap G_H \setminus \{h\}$). In order to calculate the difference between the overall cost C after selecting $h' \in S_{cand}$ as the new hub for elements in $\mathcal{C}$, we first rewrite the overall cost as follows:

$$C = \sum_{i \in G_N} (\chi O_i + \delta D_i) d_{i,A[i]} + \sum_{k \in H} \sum_{m \in H} W_H[k][m] \cdot \alpha d_{m,k}, \quad (11)$$

where the first term is the sum of the cost of collection links and distribution links, and the second term is the total cost of transfer link. In order to clarify the process of the computation, we rewrite the previous equation as follows:

$$\begin{aligned} C = & \sum_{i \in G_N \setminus \mathcal{C}} (\chi O_i + \delta D_i) d_{i,A[i]} + \sum_{k \in H \setminus \{h\}} \sum_{m \in H \setminus \{h\}} W_H[k][m] \cdot \alpha d_{m,k} \\ & + \sum_{i \in \mathcal{C}} (\chi O_i + \delta D_i) d_{i,h} + \sum_{m \in H \setminus \{h\}} W_H[h][m] \cdot \alpha d_{m,h} \\ & + \sum_{k \in H \setminus \{h\}} W_H[k][h] \cdot \alpha d_{h,k}, \end{aligned} \quad (12)$$

where the first two terms will not be affected when the hub of $\mathcal{C}$ is changed; thus, the difference between new assignment with h' as the hub of $\mathcal{C}$ and the initial assignment is calculated as follows:

$$\begin{aligned} df = & \sum_{i \in \mathcal{C}} (\chi O_i + \delta D_i) (d_{i,h'} - d_{i,h}) \\ & + \sum_{m \in H \setminus \{h\}} W_H[h][m] \alpha (d_{m,h'} - d_{m,h}) \\ & + \sum_{k \in H \setminus \{h\}} W_H[k][h] \alpha (d_{k,h'} - d_{k,h}) \\ = & \sum_{i \in \mathcal{C}} (\chi O_i + \delta D_i) (d_{i,h'} - d_{i,h}) \\ & + \alpha \sum_{k \in H \setminus \{h\}} (W_H[h][k] + W_H[k][h]) (d_{k,h'} - d_{k,h}). \end{aligned} \quad (13)$$

For cluster $\mathcal{C}$ with S_{cand} containing q_h elements, the time complexity of this local search is $O(q_h(m_h + p))$ for one cluster. W_H is generated by accessing each element in W_{in} or W_{out} exactly once; thus, the complexity of calculating is $O(pn)$. Use m_k and q_k to denote the number of grid cells and the number of elements in S_{cand} of cluster with hub k ($q_k \leq m_k$), and the overall time complexity for this local search is $O(\sum_{k \in H} q_k(m_k + p) + pn) = O(pn + \sum_{k \in H} q_k m_k)$. Alternate is also applied to network HLP in Ilić et al. (2010), and the

time complexity is $O(p \sum_{h \in H} m_h^2)$. The time complexity of applying Equation (13) to network HLP is only $O(pn + \sum_{h \in H} m_h^2)$. According to Proposition 4.1, our method is around $p/2$ times faster than previous techniques. After Alternate, the corresponding columns (the column of the original hub and the column of the chosen spoke grid cell) in W_{in} , W_{out} , and W_H should be exchanged.

Proposition 4.1. When $n \geq p^2$, $\frac{pn + \sum_{h \in H} m_h^2}{p \sum_{h \in H} m_h^2} \leq \frac{2}{p}$.

Proof. According to the Cauchy Inequality, $\frac{n^2}{p} \leq \sum_{h \in H} m_h^2 \leq n^2$ because $n = \sum_{h \in H} m_h$. Let $\sum_{h \in H} m_h^2 = \frac{n^2}{k}$, $1 \leq k \leq p$. Then $\frac{pn + \sum_{h \in H} m_h^2}{p \sum_{h \in H} m_h^2} = \frac{pn + \frac{n^2}{k}}{p \frac{n^2}{k}} = \frac{k}{n} + \frac{1}{p}$. When $n \geq p^2$, we have $\frac{k}{n} + \frac{1}{p} \leq \frac{p}{n} + \frac{1}{p} = \frac{2}{p}$. $\square$

4.3.2. Reassign. The second local search covers the reassignment of every spoke (nonhub) grid cell. In this method, the most cost-saving hub for every grid cell when other elements remain unchanged is found.

In order to calculate the difference between the overall costs after changing the assignment of one single grid cell, the overall cost C is rewritten as Equation (14). Considering reassigning grid cell g from h to h' , the initial cost is rewritten as Equation (15), where the first term and the third term will not be affected when changing the assignment of g . After changing $A[g]$ from h to h' , the difference of costs is calculated as Equation (16), and only $(2n)$ elements in W_{in} and W_{out} are changed. The difference between the two new matrices W_{in}^n and W_{out}^n and the two initial matrices are shown in Equations (17) to Equation (20).

$$\begin{aligned} C = & \sum_{i \in G_N} (\chi O_i + \delta D_i) d_{i,A[i]} \\ & + \frac{1}{2} \alpha \sum_{i \in G_N} \sum_{k \in H} (W_{in}[i][k] + W_{out}[i][k]) d_{A[i],k} \end{aligned} \quad (14)$$

$$\begin{aligned} C = & \sum_{i \in G_N \setminus \{g\}} (\chi O_i + \delta D_i) d_{A[i],i} + (\chi O_g + \delta D_g) d_{h,g} \\ & + \frac{1}{2} \alpha \sum_{i \in G_N \setminus \{g\}} \sum_{k \in H \setminus \{h, h'\}} (W_{in}[i][k] + W_{out}[i][k]) d_{A[i],k} \\ & + \frac{1}{2} \alpha \left[\sum_{i \in G_N \setminus \{g\}} (W_{in}[i][h] + W_{out}[i][h]) d_{h,A[i]} \right. \\ & + \sum_{i \in G_N \setminus \{g\}} (W_{in}[i][h'] + W_{out}[i][h']) d_{h',A[i]} \\ & + \sum_{k \in H \setminus \{h, h'\}} (W_{in}[g][k] + W_{out}[g][k]) d_{h,k} \\ & \left. + (W_{in}[g][h'] + W_{out}[g][h']) d_{h,h'} \right] \end{aligned} \quad (15)$$

$$\begin{aligned}
df = & (d_{g,h'} - d_{g,h})(\chi O_g + \delta D_g) \\
& + \frac{1}{2}\alpha \left\{ \sum_{i \in G_N \setminus \{g\}} [(W_{in}^n[i][h] + W_{out}^n[i][h]) - (W_{in}[i][h] \right. \\
& + W_{out}[i][h])d_{h,A[i]} + \sum_{i \in G_N \setminus \{g\}} [(W_{in}^n[i][h'] + W_{out}^n[i][h']) \\
& - (W_{in}[i][h'] + W_{out}[i][h'])d_{h',A[i]} \\
& + \sum_{k \in H \setminus \{h,h'\}} (W_{in}[g][k] + W_{out}[g][k])(d_{h',k} - d_{h,k}) \\
& + (W_{in}^n[g][h] - W_{in}[g][h'] + W_{out}^n[g][h] \\
& \left. - W_{out}[g][h'])d_{h,h'} \right\} \quad (16)
\end{aligned}$$

$$W_{in}^n[i][h] = W_{in}[i][h] - w_{gi} \quad \forall i \in G_N \quad (17)$$

$$W_{in}^n[i][h'] = W_{in}[i][h'] + w_{gi} \quad \forall i \in G_N \quad (18)$$

$$W_{out}^n[i][h] = W_{out}[i][h] - w_{ig} \quad \forall i \in G_N \quad (19)$$

$$W_{out}^n[i][h'] = W_{out}[i][h'] + w_{ig} \quad \forall i \in G_N \quad (20)$$

By inserting Equation (17) to Equation (20) into Equation (16), We can rewrite Equation (16) as follows:

$$\begin{aligned}
df = & (d_{g,h'} - d_{g,h})(\chi O_g + \delta D_g) + \alpha \sum_{k \in H} [(W_{in}[g][k] \\
& + W_{out}[g][k])(d_{k,h'} - d_{k,h}) - 2\alpha d_{h,h'} w_{gg}]. \quad (21)
\end{aligned}$$

If a new assignment reduces the overall cost, matrices W_{in} and W_{out} are updated as shown in Equations (17) to (20).

The time complexity for calculating the difference between two assignments is $O(p)$; therefore, the overall time complexity of this local search method is $O(np^2)$.

4.3.3. Explore. Explore is a neighborhood search that focuses on the spatial adjacency of the grid cells. In Explore, a nonhub grid cell g ($g \in G_H, g \notin H$) that is near to one cluster $\mathcal{C}$, but does not belong to it ($g \notin \mathcal{C}$) becomes the hub of this cluster. For cluster $\mathcal{C}$ with

hub h , we denote the set of all adjacent grid cells as S_{adj} and S_{cand} is equal to $G_H \cap (S_{adj} \setminus \mathcal{C})$. In Figure 1, visualization of $S_{adj} \setminus \mathcal{C}$ for clusters with three different types of structures is presented. In Alternate, we have already tested if any hubable grid cell in the cluster would be a more cost-saving hub for this cluster; thus, only hubable cells in $S_{adj} \setminus \mathcal{C}$ need to be analyzed in Explore.

This method could be regarded as a combination of Reassign and Alternate—after reassigning g to h , let g become the new hub of cluster $\mathcal{C}$. The difference between the new assignment and the initial one is calculated as follows:

$$\begin{aligned}
df = & -d_{g,h_0}(\chi O_g + \delta D_g) + \alpha \sum_{k \in H \setminus \{h\}} (W_{in}[g][k] \\
& + W_{out}[g][k]) \cdot (d_{k,g} - d_{k,h_0}) - 2\alpha w_{gg} d_{g,h_0} \\
& + \sum_{i \in \mathcal{C}} (d_{i,g} - d_{i,h})(\chi O_i + \delta D_i) + \alpha \sum_{k \in H \setminus \{h\}} (W_H[h][k] \\
& + W_H[k][h])(d_{k,g} - d_{k,h}) \\
& - \alpha (W_{in}[g][h] + W_{out}[g][h])d_{g,h_0}, \quad (22)
\end{aligned}$$

where h_0 is the previous hub of g . When the new assignment is accepted, W_{in} and W_{out} are updated as follows:

$$W_{in}^n[i][h_0] = W_{in}[i][h_0] - w_{gi} \quad \forall i \in G_N, \quad (23)$$

$$W_{out}^n[i][h_0] = W_{out}[i][h_0] - w_{ig} \quad \forall i \in G_N, \quad (24)$$

$$W_{in}^n[i][g] = W_{in}[i][h] + w_{gi} \quad \forall i \in G_N, \quad (25)$$

$$W_{out}^n[i][g] = W_{out}[i][h] + w_{ig} \quad \forall i \in G_N, \quad (26)$$

$$W_H^N[h_0][k] = W_H[h_0][k] - W_{out}[g][k] \quad \forall k \in H \setminus \{h_0, h\}, \quad (27)$$

$$W_H^N[k][h_0] = W_H[k][h_0] - W_{in}[g][k] \quad \forall k \in H \setminus \{h_0, h\}, \quad (28)$$

$$W_H^N[g][k] = W_H[h][k] + W_{out}[g][k] \quad \forall k \in H \setminus \{h_0, h\}, \quad (29)$$

Figure 1. (Color online) Visualization of $S_{adj} \setminus \mathcal{C}$ for Clusters with Different Structures; $S_{adj} \setminus \mathcal{C}$ Is the Set of the Grid Cells Filled with Dark Color for Each Cluster (Filled with Light Color): (a) $S_{adj} \setminus \mathcal{C}$ for Cluster with the Shape of a Rectangle; (b) $S_{adj} \setminus \mathcal{C}$ For Cluster with Irregular Shape; and (c) $S_{adj} \setminus \mathcal{C}$ for Cluster Contains Discrete Area

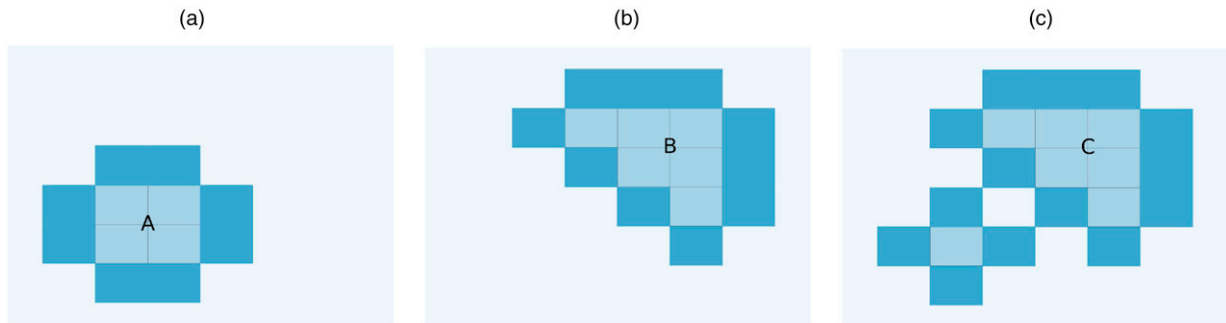
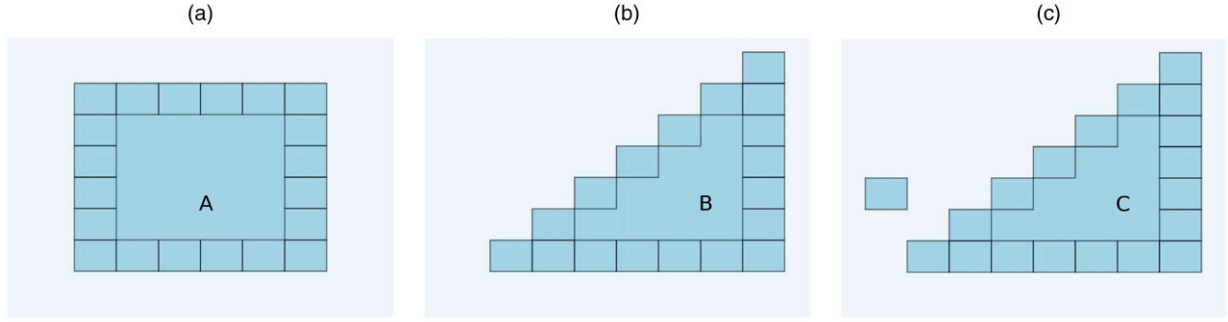


Figure 2. (Color online) Visualization of S_{bor} (the Grid Cells on the Border of Clusters) with Different Structures



Notes. Grid cells on the border are with black edges. (a) Cluster with a rectangular shape. (b) Cluster with an irregular shape. (c) Cluster contains discrete area.

$$W_H^n[k][g] = W_H[k][h] + W_{in}[g][k] \quad \forall k \in H \setminus \{h_0, h\}, \quad (30)$$

$$W_H^n[h_0][h_0] = W_H[h_0][h_0] - W_{out}[g][h_0] - W_{in}[g][h_0] + w_{gg}, \quad (31)$$

$$W_H^n[g][h_0] = W_H[h][h_0] + W_{out}[g][h_0] - W_{in}[g][h] - w_{gg}, \quad (32)$$

$$W_H^n[g][g] = W_H[h][h] + W_{out}[g][h] + W_{in}[g][h] + w_{gg}, \quad (33)$$

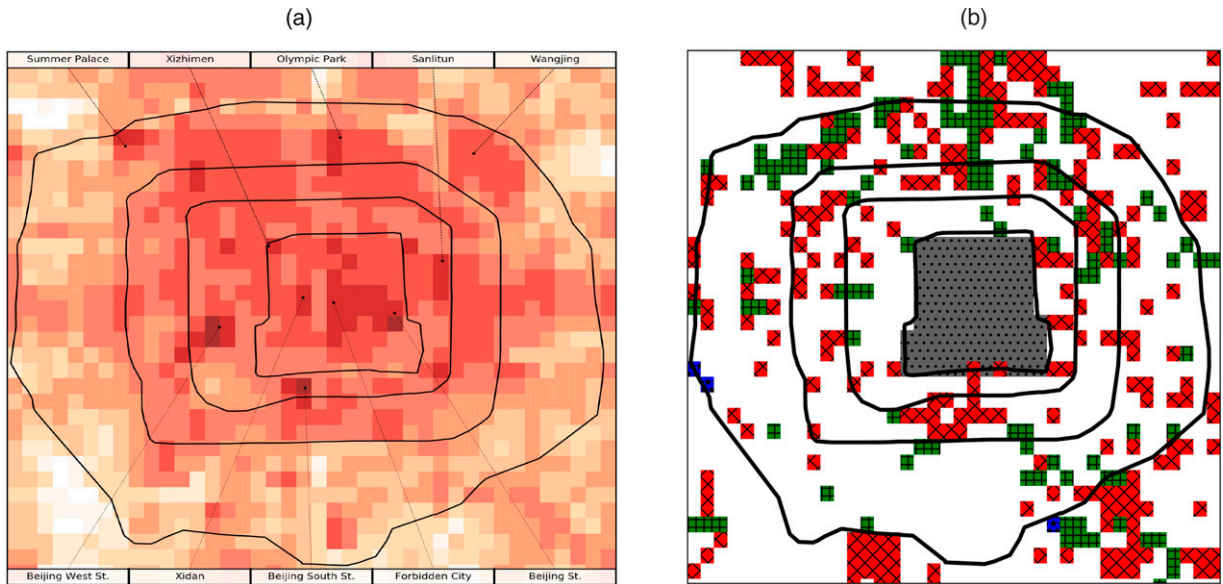
$$W_H^n[h_0][g] = W_H[h_0][h] - W_{out}[g][h] + W_{in}[g][h_0] - w_{gg}. \quad (34)$$

For cluster $\mathcal{C}$ with S_{cand} containing q_h elements, the time complexity of this local search is $O(q_h(m_h + p))$. With

the index of every grid cell, S_{cand} is generated after accessing the elements in $\mathcal{C}$ once; thus, the overall time complexity for all clusters is $O(pn + \sum_{h \in H} q_h m_h)$.

4.3.4. Substitute. In Substitute, a spoke grid cell g is selected to replace a hub h in the hub set, and then all of the grid cells in the clusters that contain h and g are allocated to the nearest hub in the new hub set (including g without h). We use M to denote the union of the cluster containing h and the cluster containing g . A is used to denote the current assignment, whereas A^* denotes the new assignment ($\forall i \in G \setminus M, A^*[i] = A[i]$). We first rewrite the overall cost C as Equation (35), and the first term in Equation (35) will not be changed after Substitute; thus,

Figure 3. (Color online) Visualization of Travel Demand (a) and Nonhubable Area of Beijing (b), with a Grid Division of 40×40



Notes. On the left, grid cells with darker color indicate larger travel demand of cells. On the right, the grid cells filled with different hatches is the nonhubable area of Beijing, including the area inside the Second Ring Road (filled with round points), leisure area (filled with "+"), districts and protected infrastructures (filled with "x"), and water area (filled with star).

the difference between the overall costs could be written as Equation (36).

$$\begin{aligned}
C &= \sum_{i \in G_N \setminus M} \sum_{j \in G_N \setminus M} w_{ij} c_{ij} + \sum_{i \in M} \sum_{j \in G_N \setminus M} w_{ij} c_{ij} \\
&+ \sum_{i \in G_N \setminus M} \sum_{j \in M} w_{ij} c_{ij} + \sum_{i \in M} \sum_{j \in M} w_{ij} c_{ij} \quad (35) \\
df &= \sum_{i \in M} \sum_{j \in G_N \setminus M} w_{ij} \left[(\chi d_{i,A^*[i]} + \alpha d_{A^*[i],A[j]}) \right. \\
&\quad \left. - (\chi d_{i,A[j]} + \alpha d_{A[i],A[j]}) \right] \\
&+ \sum_{i \in G_N \setminus M} \sum_{j \in M} w_{ij} \left[(\alpha d_{A[i],A^*[j]} + \delta d_{A^*[j],j}) \right. \\
&\quad \left. - (\alpha d_{A[i],A[j]} + \delta d_{A[j],j}) \right] \\
&+ \sum_{i \in M} \sum_{j \in M} w_{ij} \left[(\chi d_{i,A^*[i]} + \alpha d_{A^*[i],A^*[j]} \right. \\
&\quad \left. + \delta d_{A^*[j],j}) - (\chi d_{i,A[i]} + \alpha d_{A[i],A[j]} + \delta d_{A[j],j}) \right] \\
&= \sum_{i \in M} \sum_{j \in G_N} w_{ij} (\chi d_{i,A^*[i]} - \chi d_{i,A[i]}) \\
&+ \sum_{i \in G_N} \sum_{j \in M} w_{ij} (\delta d_{A^*[j],j} - \delta d_{A[j],j}) \\
&+ \sum_{i \in M} \sum_{j \in G_N \setminus M} (w_{ij} + w_{ji}) (\alpha d_{A^*[i],A[j]} \\
&\quad - \alpha d_{A[i],A[j]}) + \sum_{i \in M} \sum_{j \in M} w_{ij} (\alpha d_{A^*[i],A^*[j]} - \alpha d_{A[i],A[j]}) \\
&= \sum_{i \in M} \chi O_i (d_{i,A^*[i]} - d_{i,A[i]}) + \sum_{j \in M} \delta D_j (d_{A^*[j],j} - d_{A[j],j}) \\
&\quad - \sum_{i \in M} \sum_{j \in M} (w_{ij} + w_{ji}) (\alpha d_{A^*[i],A[j]} - \alpha d_{A[i],A[j]}) \\
&\quad + \sum_{i \in M} \sum_{j \in G_N} (w_{ij} + w_{ji}) (\alpha d_{A^*[i],A[j]} - \alpha d_{A[i],A[j]}) \\
&\quad + \sum_{i \in M} \sum_{j \in M} w_{ij} (\alpha d_{A^*[i],A^*[j]} - \alpha d_{A[i],A[j]}) \\
&= \sum_{i \in M} (\chi O_i + \delta D_i) (d_{i,A^*[i]} - d_{i,A[i]}) \\
&\quad + \alpha \sum_{i \in M} \sum_{k \in H} (W_{out}[i][k] + W_{in}[i][k]) (d_{A^*[i],k} - d_{A[i],k}) \\
&\quad + \alpha \sum_{i \in M} \sum_{j \in M} w_{ij} (d_{A^*[i],A^*[j]} + d_{A[i],A[j]} \\
&\quad - d_{A^*[i],A[j]} - d_{A[i],A^*[j]}) \quad (36)
\end{aligned}$$

Algorithm 1 (Substitute)

Input: W_{in} , W_{out} , $assi$

Output: W_{in} , W_{out} , $assi$, DF

- 1: Randomly select p spoke hubable grid cells as $list_g$
- 2: $DF = 0$, Set H_1 as current hub set
- 3: **for** every g in $list_g$ **do**
- 4: $h_0 = assi[g]$
- 5: **for** every h in H_1 **do**
- 6: $M = \mathcal{C}_h \cup \mathcal{C}_{h_0}$

- 7: Let H_2 be the current hub set, remove h from H_2 and add g to H_2
- 8: Assign every i in M to the nearest hub in H_2 and get $assi_{temp}$
- 9: Calculate df between $assi_{temp}$ and $assi$ using Equation (36)
- 10: **if** $df < 0$ **then**
- 11: Remove h from H_1
- 12: **exit**
- 13: **end if**
- 14: **end for**
- 15: **if** $df < 0$ **then**
- 16: $DF += df$
- 17: **for** g in M **do**
- 18: **for** i in N **do**
- 19: **if** $assi_{temp}[i] \neq assi[i]$ **then**
- 20: $W_{in}[i][assi_{temp}[g]] += w_{gi}$
- 21: $W_{in}[i][assi[g]] -= w_{gi}$
- 22: $W_{out}[i][assi_{temp}[g]] += w_{ig}$
- 23: $W_{out}[i][assi[g]] -= w_{ig}$
- 24: **end if**
- 25: **end for**
- 26: **end for**
- 27: $assi = assi_{temp}$
- 28: **end if**
- 29: **end for**

The time complexity of Substitute for M with m elements is $O(m^2 + mp)$. The algorithm when applying Substitute to all clusters is presented in Algorithm 1. In Algorithm 1, a subset of hubable spoke grid cells is randomly selected as $list_g$, and for every spoke grid cell in $list_g$, we test if the solution is improved after replacing g with every hub h in the hub set and accept a better solution. For each new grid cell, we need to test it with every old hub; thus, there is an inner loop (for every old hub) and an outer loop (for each new grid cell). When the new assignment is accepted, W_{in} and W_{out} are updated using the same equations discussed in Reassign, because we can regard Substitute as reassigning all of the grid cells in M to a new hub. The total time complexity of this local search is $O(p \sum_{h \in H} m_h^2)$ without considering the update of W_{in} and W_{out} .

4.3.5. SwapK. In SwapK, k hubs in the hub set are replaced by k hubable spoke grid cells, and Reassign is applied for the new set of spoke grid cells using the new set of hubs. For example, in Swap1 ($k = 1$), a hub g replaces hub h' in the hub set, and we then apply Reassign to all the current spoke grid cells (including h without g) using the current hub set (including g without h). We denote the set of spoke grid cells that we selected as $list_g$ and the set of hubs is going to be replaced as $list_h$. The difference of SwapK can be calculated using two steps. (1) We set each g as a new

hub by assigning g as the new hub of the cluster that h previously belongs to. In this step, the difference of the cost can be calculated using the delta function of Explore (if g and h are in two clusters)/Alternate (if g and h are in the same cluster) for a specific node, and update W_{in} and W_{out} . Therefore, the complexity of the this step is $O(\sum_{h \in list_h} m_h + nk)$, where m_h is the number of elements in cluster h . (2) Reassign is applied using the current hub set including $list_g$ but excluding $list_h$ ($H \setminus list_h \cup list_g$). Generally, when Reassign is used for a grid cell, it calculates the difference of cost by assigning the grid cell g to each hub in the hub set (H) and chose the hub with the smallest cost as the new hub for g . Here, each grid cell g is allocated to each hub in the current hub set ($H \setminus list_h \cup list_g$) and chooses the hub with the smallest cost. The complexity of this step is exactly the same as Reassign, because only the elements in the hub set are different. The whole process of SwapK is presented in Algorithm 2. Given that the time complexity of Step 1 is only $O(\sum_{k \in list_h} m_h + kp)$, whereas the time complexity of Step 2 is $O(np^2)$, the time complexity of SwapK is almost the same as Step 2, which is the same as Reassign.

Algorithm 2 (SwapK)

Input: $W_{in}, W_{out}, assi, k$
Output: $W_{in}, W_{out}, assi, DF$

- 1: $df_1 = 0$
- 2: randomly select k spoke grid cells as $list_g$, k hub nodes as $list_h$
- 3: let S be the set of current spoke grid cell, let H be the set of current hubs
- 4: $H' = (list_g \cup H) \setminus list_h; S' = (list_h \cup S) \setminus list_g$
- 5: **for** $i < k$ **do**
- 6: $h = list_h[i], g = list_g[i]$
- 7: calculate df_i using Equation (13) or Equation (22)
- 8: $df_1 += df_i$
- 9: update W_{in} and W_{out}
- 10: $i++ = 1$
- 11: **end for**
- 12: $DF = df_1$
- 13: **for every** g in S' **do**
- 14: $df_g = inf$
- 15: **for every** h in H' **do**
- 16: calculate df_{temp} using Equation (21)
- 17: **if** $df_{temp} < df_g$ **then**
- 18: $df_g = df_{temp}$
- 19: $sel_h = h$
- 20: **end if**
- 21: **end for**
- 22: **if** $df_g < 0$ **then**
- 23: $assi[g] = sel_h$, update W_{in} and W_{out} using Equations (17) to (20)
- 24: $DF += df_g$
- 25: **end if**
- 26: **end for**

4.4. Shake

In Shake, a random solution in the neighborhood of the best assignment thus far is generated. Given integer r , r hubs in H are exchanged with r nonhub grid cells. Shake is performed by sequentially exchanging one hub and one spoke grid cell for r times. When h , the hub of $\mathcal{C}$ and a hubable spoke grid cell g is exchanged, h becomes a spoke grid cell in the cluster that g previously was, and g becomes the hub of $\mathcal{C}$. When g and h are in the same cluster, this process could be regarded as Alternate. If g and h belong to different clusters, this could be regarded as a sequence of Explore for $\mathcal{C}$ and Reassign for one grid cell h —after setting g as the hub of $\mathcal{C}$, reassign h to h_0 which is the previous hub of g . The difference is calculated as follows:

$$\begin{aligned}
 df = & -d_{g,h_0}(\chi O_g + \delta D_g) + \alpha \sum_{k \in H \setminus \{h\}} (W_{in}[g][k] \\
 & + W_{out}[g][k]) \cdot (d_{k,g} - d_{k,h_0}) - 2\alpha w_{gg} d_{g,h_0} \\
 & + \sum_{i \in \mathcal{C} \setminus \{h\}} (d_{i,g} - d_{i,h})(\chi O_i + \delta D_i) \\
 & + \alpha \sum_{k \in H \setminus \{h\}} (W_H[h][k] + W_H[k][h])(d_{k,g} - d_{k,h}) \\
 & - \alpha (W_{in}[g][h] + W_{out}[g][h])d_{g,h_0} + d_{h,h_0}(\chi O_h + \delta D_h) \\
 & - 2\alpha d_{g,h_0} w_{hh} + 2\alpha (w_{gh} + w_{hg})d_{g,h_0} \\
 & + \alpha (W_{in}[h][h] + W_{out}[h][h])d_{g,h_0} \\
 & + \alpha \sum_{k \in H \setminus \{h\}} ((W_{in}[h][k] + W_{out}[h][k])(d_{k,h_0} - d_{k,g})).
 \end{aligned} \tag{37}$$

W_{in} and W_{out} are updated by sequentially applying the update techniques introduced in Explore (Equation (23)–(26)) and Reassign (Equation (17)–(20)). The time complexity is $O(p + m)$ for Equation (37) and $O(rp + \sum_{i < r} m_i)$ for Shake, where m_i is the number of elements of $\mathcal{C}_i$.

4.5. Variable Neighborhood Descent

Variable neighborhood descent (VND) that performs changes of neighborhoods in a deterministic way is an important variant of VNS. Sequential VND that performs local searches sequentially is commonly used, but it can only explore a small neighborhood. In Ilić et al. (2010), three local searches were applied, and they developed a reduced nested strategy called Mix-VND that performs first two local searches sequentially for each point that belongs to the neighborhood of the last local search. According to the results, Mix-VND obtained better solutions with less computation time than sequential on larger Planet instances. Therefore, we apply Mix-VND in our heuristic. In

the nested layer of the Mix-VND in Ilić et al. (2010), they only explored one neighborhood. In order to explore a larger neighborhood, we propose a new Mix-VND called Multi-Mix-VND, which sequentially explores the first to the $(l_{nest} - 1)th$ ($l_{nest} > 1$) neighborhood in the nested layer. Multi-Mix-VND is given in Algorithm 3. When l_{nest} equals one, Multi-Mix-VND is equivalent to Mix-VND.

Algorithm 3 (Multi-Mix-VND)

Input: initial solution x , objective of x : $f(x)$

- 1: Determine the order of l_{nest} local searches used in the nested layer and l_{seq} local searches used in the sequential layer
- 2: $l_1 = 0, l_2 = 0 // l_1$: current local search used in the nested layer; l_2 : current local search used in the sequential layer.
- 3: **while** $l_1 < l_{nest}$ **do**
- 4: **for every** x_1 in $N_{l_1}(x)$ **do**
- 5: //for every point in the l_1 th neighborhood
- 6: **while** $l_2 < l_{seq}$ **do**
- 7: $f^n \leftarrow \min_{x_2 \in N_{l_2}(x_1)} f(x_2)$ //find the best solution in $N_{l_2}(x_1)$
- 8: $x^n \leftarrow \operatorname{argmin}_{x_2 \in N_{l_2}(x_1)} f(x_2)$
- 9: $l_2 + = 1$
- 10: **if** $f^n < f$ **then**
- 11: $f \leftarrow f^n, x \leftarrow x^n, l_2 \leftarrow 1$
- 12: **end if**
- 13: **end while**
- 14: **end for**
- 15: $l_1 + = 1$
- 16: **end while**

4.6. Overall VNS-Based Heuristic

Algorithm 4 (Grid-VNS)

- 1: Set nonimprove_max , k_{max} let $\text{nonimprove_num} = 0$, $\text{iter_num} = 0$
- 2: Generate initial assignment A_{ini}
- 3: Let $A = A_{ini}$, compute the objective of Assignment($\text{obj}(A)$), Let $\text{best_obj} = \text{obj}(A)$
- 4: **while** $\text{nonimprove_num} < \text{nonimprove_max}$ **do**
- 5: Let $A_c = \text{Shake}(A, \min\{k_{max}, p\})$
- 6: Let $j = 1, k = 1$, and $\text{obj_c} = \text{obj}(A_c)$
- 7: **while** $k < k_{max}$ **do**
- 8: randomly select num pairs (each denoted as H_{out}, NH_{in}), containing k elements, respectively, as the replace_list .
- 9: **for every** pair of (H_{out}, NH_{in}) in replace_list . **do**
- 10: $A_c = \text{SwapK}((H_{out}, NH_{in}))$ $D_obj_0 = \text{df}(A_c)$ //compute the difference of the assignments
- 11: **while** $j < 6$ **do**
- 12: **if** $j == 1$ **then**
- 13: $A_c = \text{Alternate}(A_c), D_obj = \text{df}(A_c)$
- 14: **end if**

- 15: **if** $j == 2$ **then**
- 16: $A_c = \text{Reassign}(A_c)(\text{partial}), D_obj = \text{df}(A_c)$
- 17: **end if**
- 18: **if** $j == 3$ **then**
- 19: $A_c = \text{Explore}(A_c), D_obj = \text{df}(A_c)$
- 20: **end if**
- 21: **if** $j == 4$ **then**
- 22: $A_c = \text{Reassign}(A_c)(\text{complete}), D_obj = \text{df}(A_c)$
- 23: **end if**
- 24: **if** $j == 5$ **then**
- 25: $A_c = \text{Substitute}(A_c), D_obj = \text{df}(A_c)$
- 26: **end if**
- 27: $j + = 1$
- 28: **if** $D_obj_0 + D_obj < 0$ **then**
- 29: $\text{obj_c} + = D_obj, j = 1$
- 30: **end if**
- 31: **end while**
- 32: $\text{iter_num} + = 1$
- 33: **if** $\text{obj_c} < \text{best_obj}$ **then**
- 34: $k = 0, \text{best_obj} = \text{obj_c}, A = A_c, \text{nonimprove_num} = 0, \text{exit}$
- 35: **else**
- 36: $\text{nonimprove_num} + = 1$
- 37: **end if**
- 38: **end for**
- 39: **if** $k == 0$ **then**
- 40: exit
- 41: **else**
- 42: $k + = 1$
- 43: **end if**
- 44: **if** $\text{nonimprove_num} > \text{nonimprove_max}$ **then**
- 45: exit
- 46: **end if**
- 47: **end while**
- 48: **if** $\text{nonimprove_num} > \text{nonimprove_max}$ **then**
- 49: exit
- 50: **end if**
- 51: **end while**

In our algorithm, we first generate an initial assignment and then search the current neighborhood using Multi-Mix-VND and move to the neighborhood when a better solution is found. Shake operation is applied after searching the whole neighborhood of the current assignment to jump out of local minima. Every time we apply Shake, we exchange k_{max} hubs with k_{max} hubable spoke grid cells. If no improvement is found for nonimprove_max time, the algorithm will terminate. The heuristic is shown in Algorithm 4.

We use SwapK with k_2 ranging from 1 to $k_{max} - 1$ at the nested layer and apply Alternate, Reassign (partial), Explore, Reassign (complete), and Substitute sequentially. Given that we tested changing k hubs where k takes value between 1 to $k_{max} - 1$ in the SwapK

local search, we only change k_{max} hubs in the shake procedure. Swap1 and Substitute both replace one hub with one spoke grid cell. However, in Swap1, the assignment of all of the spoke grid cells may change while in Substitute, only the assignment of the clusters containing the replaced hub and the candidate hub cell will change. Besides, in Substitute, grid cells are assigned to the nearest hub, whereas in Swap1, grid cells are not necessarily assigned to the nearest hub.

To accelerate the runtime, in the second local search when sequentially exploring the neighborhood, we only apply Reassign to the spoke grid cells that are on the border of every cluster (the set of spoke grid cells on the border is denoted as S_{bor}). A spoke grid cell g inside a cluster ($g \in \mathcal{C} \setminus S_{bor}$) is assigned to the same hub as all of its adjacent grid cells are allocated to, and it is very likely its current assignment is the optimal when other elements remain unchanged; thus, we only analyze the assignment of grid cells on the border (S_{bor}) in this local search. The visualization of S_{bor} is shown in Figure 2. For a cluster with a shape of an $a \times b$ rectangle, Reassign (partial) is applied for only $2(a + b - 2)$ grid cells, whereas Reassign (complete) is for $a \times b$ grid cells. To avoid missing the optimal solution, we apply a complete Reassign in the forth neighborhood search in the sequential layer.

The whole neighborhood of SwapK is very large, and we do not explore the whole neighborhood. Instead we randomly select num pairs, each denoted as (H_{out}, NH_{in}) , containing k elements, respectively, as the *replace_list*. In each execution of SwapK, one pair (H_{out}, NH_{in}) is selected. The number of solutions contained in the reduced SwapK neighborhood search is exactly num , the number of pairs of (H_{out}, NH_{in}) are selected. Every time a better solution is found when sequentially performing the local searches, the algorithm will return to the first neighborhood search Alternate.

The order of local searches in the sequential layer is decided according to their time complexity, and the simplest one is set as the first local search. Given that the local search front will be visited more often than local search behind, setting the simple one in front of the sequential layer can generate more efficient heuristics.

We use five local searches in our heuristic: Reassign, Alternate, Explore, Substitute, and SwapK. Reassign and Alternate have been studied in the literature. However, we improve the calculation of the delta function of Alternate around $p/2$ times, where p is the number of hubs. Explore is specially designed for the grid-based model, and it explores the spatial information. It is novel in the literature. Substitute is alike with Locate by Ilić et al. (2010) because they both exchange one hub node with one spoke node. In the

literature, when applying Locate with a spoke node g and a hub node h , only the assignment of cluster h has been changed. Although in Substitute, the assignment of cluster that g was in, it is also changed. SwapK is also novel in this field. It is alike with Shaking procedure by Ilić et al. (2010), given that they both change k hubs from the current hub set. However, for a single iteration of Shaking, the time complexity is $O(n^2)$, whereas it is $O(np^2)$ for SwapK. Given that $O(np^2) < O(n^2)$ in most cases, SwapK is able to visit the neighborhood with k hubs different from the current hub set in a more efficient way.

Although we use more local searches in the sequential layer than in the literature, the time complexity of our sequential layer is still comparable with theirs, given that the time complexity of our last two local searches and the two local searches they applied is the same and our first three local searches are with much lower time complexity compared with the last two local searches. Because we simplify the calculation of Alternate and use the spatial property of this problem to apply Explore and Reassign(partial), our heuristic is able to explore a larger neighborhood in comparable runtime.

We remove local searches Explore and Reassign (partial) that are specifically designed for the grid-based models from the heuristic and construct a new VNS for general USApHMP. This VNS is named as New-VNS (N-VNS for short) and is composed of four local searches: Reassign, Alternate, Substitute, and SwapK. As discussed before, Reassign and Alternate have been used in the literature, whereas Substitute and SwapK are novel in solving this problem. The VND and the whole procedure of N-VNS is exactly the same as Grid-VNS, except that there are only four local searches in N-VNS. The performance of N-VNS is shown in the online appendix.

5. Experimental Evaluation

In this section, we first present the underlying data and the experimental setup of this study. Then we report results of the preliminary experiments. Furthermore, the performance of Grid-VNS is analyzed in details. Finally, results of the Beijing instances are provided and analyzed. Problem data and results are available at <https://github.com/ScalableVertiportLocationProblem/ScalableVertiportHLP>.

5.1. Data Set and Experimental Setup

The data set containing 185,077 trip records of Beijing is used as our case study for our grid-based model. The area we analyze is a 30×37.80 -km rectangle, covering the whole area of Beijing within Fifth Ring. The distance between each pair of grid cells is the great circle distance between the centers of them. Each

record in the data set includes the longitude and latitude of the origination and destination of the trip. The distribution of travel demand in Beijing with a grid division of 40×40 is shown in Figure 3(a). In the grid cell-based framework, we are able to use a set of cells to represent the nonhubable area. As shown in Figure 3(b), we use the hatched area to represent the nonhubable area including the one inside the Second Ring Road (filled with round points), leisure area including parks (filled with “+”), districts and protected infrastructures (filled with “x”), and water area (filled with star).

For the test case of Grid-VNS, k_{max} was set as five as a balance of intensification and diversification; χ, α , and δ were set as 1.0, 0.5 and 1.0, respectively. All experiments were carried out on a computer with Intel(R) Core(TM) i7-8550u CPU of 4 GHz and 64 GB RAM, running Fedora 30.

5.2. Preliminary Experimentation

In this section, we first test the number of iterations that the heuristic need to take to generate the best-known solutions. Then we analyze the contribution of each component of the whole algorithm. Finally, we demonstrate the efficiency of implementing the delta calculation of the objective function.

5.2.1. Termination Criterion. In the first series of experimental results, we test the number of iterations that the heuristic needs to take to generate the best-known solutions. The best-known solutions are generated by Grid-VNS by setting the nonimprovement_max as a huge number ($10np$ in practical). We run each instance 10 times and document the number of iterations including the iter_num and nonimprove_num. The

maximum number of iterations is reported with a prefix Max and the average number of iteration over 10 runs is with prefix Avg in Table 2. Based on the results, we believe that setting nonimprove_max as $np/5$ is large enough, given that it is larger than Max_nonimprove_num for every instance we tested.

5.2.2. Contribution of Local Searches. To better understand the contribution of each component of the whole algorithm, we first use the distance-to-target diagram (Hansen and Mladenović 1997, Mladenović et al. 2012). In the distance-to-target diagram, point (u, v) denotes each local optima found by different local searches. u represents the Hamming distance (the number of different elements) between the local optima and the best-known solution (the origin) and V represents the % deviation of local optima value and the best-known objective. Points with a low deviation and short distance to the best-known solution are near the origin. By randomly starting an algorithm r times and draw the r solution as r points in the diagram (points of the same solutions will coincide), we can clearly see the distribution of the final solutions generated and test whether the algorithm is robust or not. We test six VND procedures, including Grid-VNS and Grid-VNS, excluding each local search. Each VND procedure is randomly started 1,000 times. As shown in Figure 4, (a)–(f), we observe that

1. The six VNDs are all able to generate near-optimal or optimal solutions. Among them, Grid-VNS performs the best in terms of solution quality.

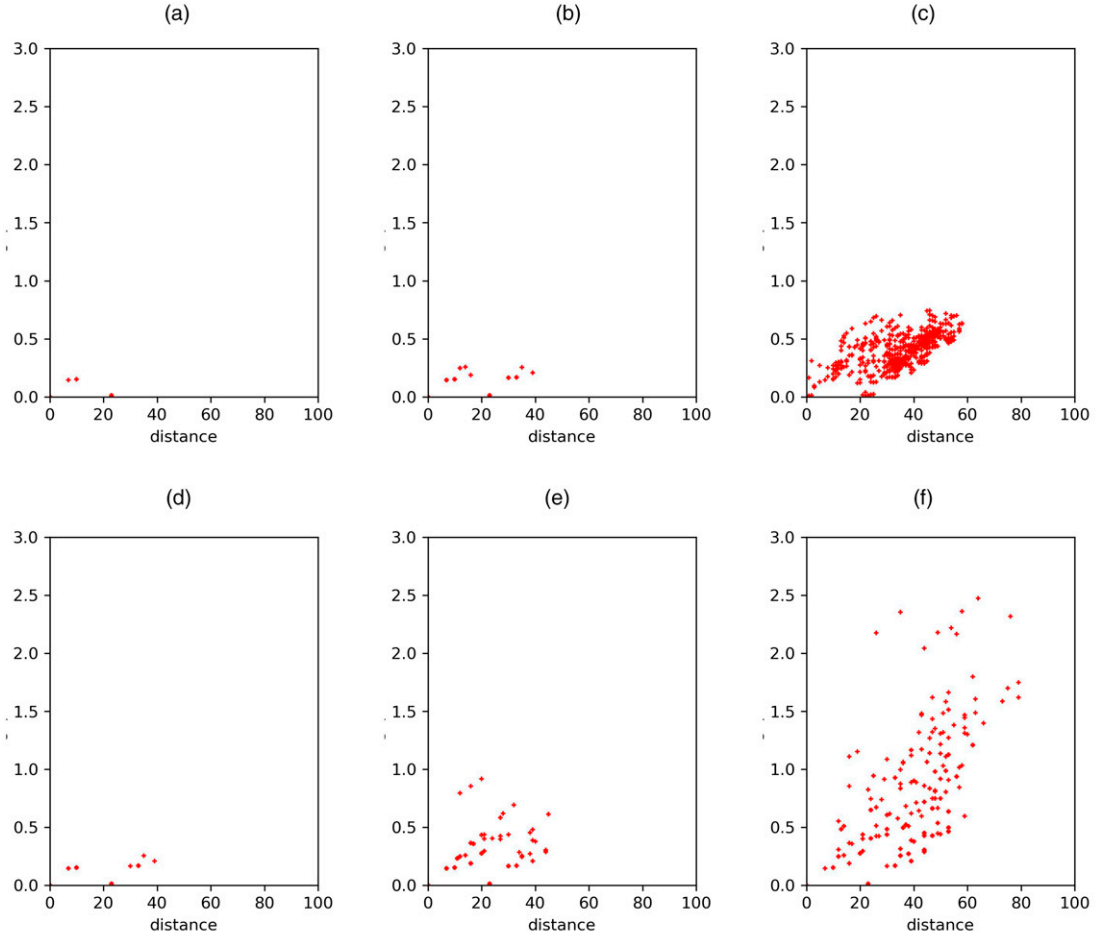
2. The difference between the six VNDs shows the importance of each local search, Alternate, Reassign, Explore, SwapK, and Substitute, respectively, because

Table 2. Number of Iterations Grid-VNS Needs to Take to Generate the Best-Known Solutions

n_b	n	p	Max_iter_num	Avg_iter_num	Max_nonimprove_num	Avg_nonimprove_num
15	225	2	9	2.8	7	0.9
		5	63	19.1	57	11.9
		10	370	115.1	307	78.1
		15	392	142.6	345	94.7
20	399	2	6	2.6	2	0.4
		5	144	55.1	109	41.9
		10	623	244.5	442	163.6
		15	2259	597.2	865	353.3
25	617	2	44	13.9	35	9.5
		5	134	53.5	90	40.5
		10	497	193.7	446	141.0
		15	919	389.6	438	222.7
30	875	2	31	11.3	15	5.5
		5	99	35.7	85	21.8
		10	948	465.9	842	338.1
		15	192	78.1	145	39.8

Note. All instances are run for 10 times.

Figure 4. (Color online) Distance-to-target Diagram of Local Searches and Different VNDs.



Notes. Each local search and VND starts 1,000 times randomly for Beijing with $n_b = 10$ and $p = 10$. All of the gaps are expressed as a percentage. (a) Grid-VNS. (b) Grid-VNS excluding Alternate. (c) Grid-VNS excluding Reassign. (d) Grid-VNS excluding Explore. (e) Grid-VNS excluding Substitute. (f) Grid-VNS excluding SwapK.

that by excluding any local search out of the heuristic, the performance of the heuristic worsens.

To comprehensively analyze the contribution of the components of our algorithm, we test the six VND procedures on instances of different sizes within the 7,200-second time limit. Each instance is run 10 times. Table 3 reports the number of times that the algorithm reaches the best-known solution (No. of Best) and the average run time (Avg_time). For instances with relatively small size, Grid-VNS, Grid-VNS excluding Explore, Grid-VNS excluding Substitute, and Grid-VNS excluding SwapK are able to generate optimal results in a relatively short time. Although Grid-VNS excluding Alternate and Grid-VNS excluding Reassign can obtain the best-known solutions, they take much more time than the other four VNDs do. For large cases, Grid-VNS outperforms the other five VNDs, in terms of solution quality and run time. Although the time limit is set as

quite long, VNDs except for Grid-VNS excluding Explore are not able to generate as good solutions as Grid-VNS. Grid-VNS excluding Explore performs about the same with Grid-VNS for small p instances but worst than Grid-VNS for large p instances.

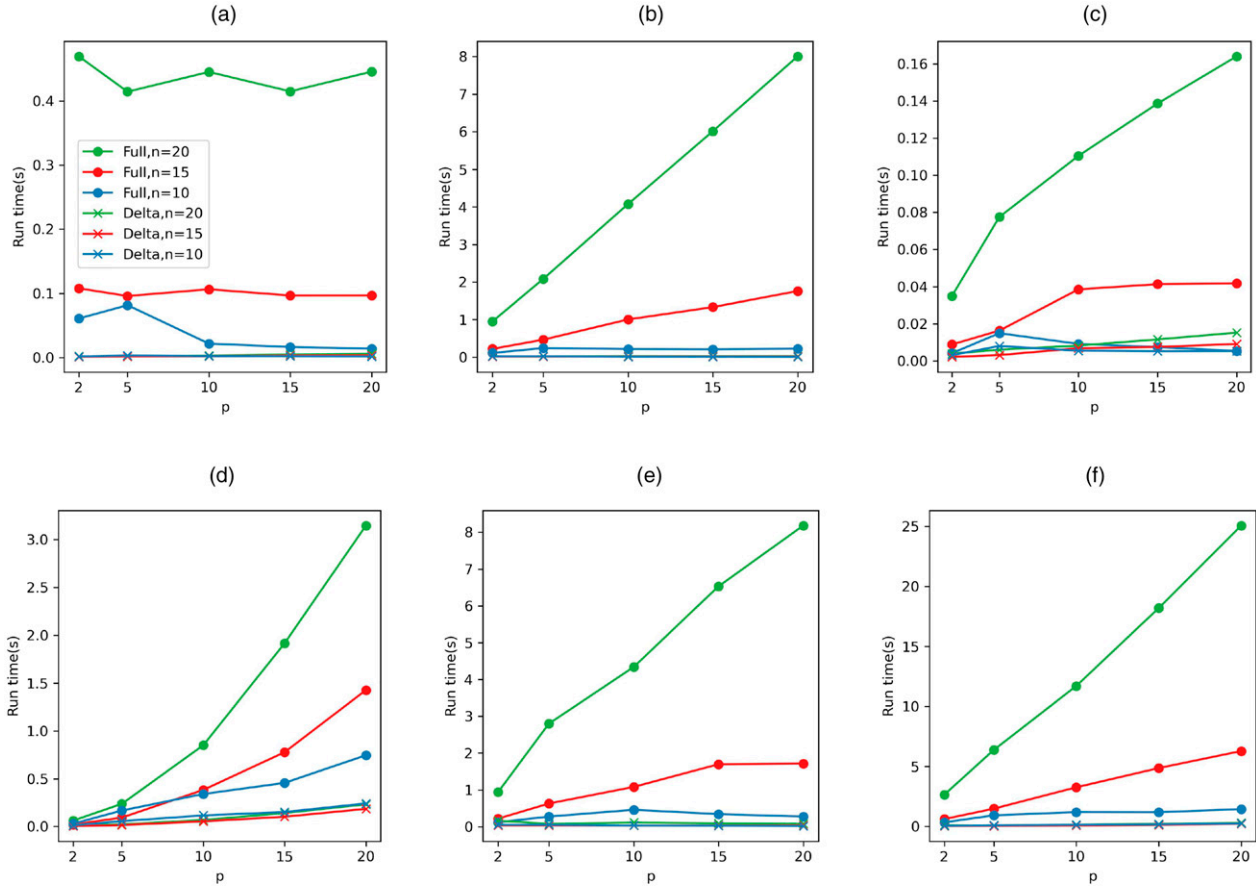
5.2.3. Importance of Delta Calculation. To demonstrate the efficiency of implementing the delta calculation of the objective function, we use the complete calculation formula ($C = \sum_{i \in G_N} \sum_{j \in G_N} w_{ij} (\chi d_{i,A[i]} + \alpha d_{A[i],A[j]} + \delta d_{A[j],j})$) to replace all of the delta function in each local search and compare the runtime. The results are shown in Figure 5, (a)–(f). For each local search, the delta function of each local search is efficient compared with full operation. For the sum of the runtime of all local searches, the delta function is 5–70 times faster than complete calculation, thus making it more efficient to explore neighborhoods in a much shorter time than

Table 3. Comparison Between the Four VND Procedures

n_b	p	Grid-VNS		Grid-VNSEXcludingAlternate		Grid-VNSEXcludingReassign		Grid-VNSEXcludingExplore		Grid-VNSEXcludingSubstitute		Grid-VNSEXcludingSwapK	
		Avg_time	No. of best	Avg_time	No. of best	Avg_time	No. of best	Avg_time	No. of best	Avg_time	No. of best	Avg_time	No. of best
15	2	0.48	10	32.79	10	2,262.51	10	1.23	10	0.45	10	0.83	10
5	5	10.36	10	36.97	10	7,200.90	3	12.46	10	10.74	10	28.52	10
10	10	118.34	10	548.33	10	7,203.01	1	416.95	10	388.43	9	326.78	10
15	15	267.14	10	673.10	10	7,204.55	0	261.39	10	1,182.39	9	968.23	10
20	2	2.16	10	28.36	10	7,200.27	7	2.44	10	2.63	10	1.78	10
5	5	49.53	10	2,097.18	10	7,200.95	0	59.17	10	70.25	10	48.64	10
10	10	198.92	10	5,374.19	3	7,204.11	0	425.36	10	1,006.02	10	729.09	10
15	15	1,692.51	10	7,203.29	0	7,202.23	0	5,185.99	7	6,465.76	2	5,217.20	6
25	2	21.87	10	598.57	10	7,200.88	0	13.13	10	6.99	10	11.77	10
5	5	104.79	10	4,106.63	8	7,203.20	0	94.23	10	64.87	10	210.71	10
10	10	359.51	10	7,202.45	0	7,205.51	0	424.38	10	981.58	9	1,096.47	10
15	15	1,325.31	10	6,689.68	2	7,205.48	0	1,500.24	10	6,171.61	3	6,130.65	3
30	2	25.44	10	3,123.87	9	7,202.09	0	15.72	10	13.03	10	8.87	10
5	5	125.22	10	6,637.18	1	7,206.88	0	115.65	10	161.40	10	181.51	10
10	10	2,018.91	10	7,205.22	0	7,208.08	0	3,077.71	8	2,683.26	3	6,931.50	1
15	15	314.96	10	7,205.37	0	7,207.20	0	268.61	10	4,172.45	5	6,688.00	9

Notes. Time limit is 7,200 seconds. The number of times that the algorithm reaches the best-known solution is reported as no. of best and the average run time as Avg_time.

Figure 5. (Color online) Comparison Between the Delta Function and the Full Operation (Recalculate the Objective Function Completely) on Beijing Instances with $p \in \{2, 5, 10, 15, 20\}$ and $n_b \in \{10, 15, 20\}$



Notes. (a) Alternate. (b) Reassign. (c) Explore. (d) Substitute. (e) SwapK. (f) Total (sum of the runtime of all local searches).

recalculating the objective function. Figures 4 and 5 show that our good results lie in both the efficient implementation of delta function and the design of the neighborhood structure.

5.3. Comparison with Other Approaches

In order to evaluate the performance of our VNS heuristic, comparisons with several methods including CPLEX, LocalSolver, Mix-GVNS, N-VNS, and Iterative Weiszfeld-type Algorithm (IWA) are performed. The termination rule for Mix-GVNS was $n/2$ without improvement, which is the same as those provided in Ilić et al. (2010). In order to apply IWA, we rewrite the overall travel cost as follows:

$$C = \sum_{1 \leq i < j \leq p} \alpha \|H_i - H_j\| (W_H[i][j] + W_H[j][i]) + \sum_{g \in G_N} \|g - A[g]\| (\chi O_g + \delta D_g), \quad (38)$$

where $\|H_i - H_j\|$ denotes the Euclidean distance between hub i and hub j , and $\|g - A[g]\|$ denotes the Euclidean distance between grid cell g and its hub $A[g]$.

When assuming every grid cell is allocated to the nearest hub, Weiszfeld's algorithm (Eyster et al. 1973) can be applied. For a fair comparison, we replaced every hub location generated by IWA with the nearest center of grid cell in G_H and recomputed the overall cost.

5.3.1. Comparison Between Grid-VNS with CPLEX and LocalSolver.

We compared the standard software CPLEX and heuristic-based solver LocalSolver with our proposed grid-based heuristic. Given that the runtime of CPLEX increases significantly as n_b becomes larger, we only performed experiments for instances with $n_b \leq 12$. The detailed results and comparison with large p instances are shown in Table 4. For each instance, we report the value of the objective function obtained by CPLEX with a time limit of 36,000 seconds and the runtime needed to reach this objective. The time limit is set as 3,600 seconds for LocalSolver. When performing Grid-VNS, every instance was run 10 times and we report the best objective value found in these 10 times as Best_Obj in the table. The average deviation compared with the results generated by CPLEX is presented as Avg_Gap. Grid-VNS found

Table 4. Performance of CPLEX and Grid-VNS (10 Times)

n_b	n	n_h	p	CPLEX			CPLEX2		LocalSolver		Grid-VNS		
				Obj	Time1	Time2	Gap	Time1	Gap	Time1	Best_obj	Time1	Avg_gap
4	16	11	2	3,025,048.5	0.1	0.1	0	0.1	0	0.9	3,025,048.5	0.0	0
5	25	19	2	3,216,738.8	0.2	0.8	0	0.5	0	29.7	3,216,738.8	0.1	0
6	36	27	2	2,868,937.5	6.7	7.2	0	1.4	0	99.0	2,868,937.5	0.2	0
7	49	34	2	3,137,937.2	5.9	6.8	0	2.8	0	632.8	3,137,937.2	0.1	0
8	64	42	2	3,335,882.2	59.6	65.9	0	6.6	0	2,667.2	3,335,882.2	0.6	0
9	81	57	2	2,919,133.6	149.7	156.3	0	21.3	—	3,626.9	2,919,133.6	0.1	0
10	100	67	2	3,034,998.7	1,640.0	2,426.9	0	71.8	—	3,645.9	3,034,998.7	0.6	0
11	121	85	2	2,954,303.7	1,908.7	4,243.6	0	166.5	—	3,683.8	2,954,303.7	0.4	0
12	144	96	2	2,899,817.3	11,811.3	13,140.5	0	313.5	—	3,731.0	2,899,817.3	0.2	0
6	36	27	5	2,186,158.0	2.2	2.6	0	2.0	0	64.8	2,186,158.0	0.2	0
7	49	34	5	2,506,851.9	4.6	7.4	0	5.0	0	876.4	2,506,851.9	0.4	0
8	64	42	5	2,614,842.4	210.8	221.6	0	15.0	—	3,613.7	2,614,842.4	0.5	0
9	81	57	5	2,366,864.9	703.5	913.3	0	33.5	—	3,626.5	2,366,864.9	1.6	0
10	100	67	5	2,462,767.0	3,121.1	3,150.9	0	82.9	—	3,642.0	2,462,767.0	0.6	0
11	121	85	5	2,432,875.2	13,634.1	36,043.9	0	214.8	58.429	3,682.6	2,432,875.2	4.5	0
12	144	96	5	2,390,949.9	28,386.4	36,148.8	−0.642	381.0	—	3,724.9	2,375,591.9	1.7	−0.642
8	64	42	10	2,292,486.6	19.2	72.5	0	19.7	—	3,614.1	2,292,486.6	1.5	0
9	81	57	10	2,032,507.7	2,873.1	2,931.3	0	72.3	—	3,624.9	2,032,507.7	16.0	0
10	100	67	10	2,211,559.1	4,953.2	36,120.9	0	207.1	—	3,644.5	2,211,559.1	40.8	0
11	121	85	10	2,118,538.2	18,946.7	36,040.1	−0.019	933.4	—	3,680.2	2,118,132.8	7.0	−0.019
12	144	96	10	2,101,741.7	33,194.3	36,127.4	−0.737	766.1	—	3,729.4	2,086,256.7	27.3	−0.737
10	100	67	15	2,057,327.8	3,344.5	36,013.3	0	294.6	—	3,645.5	2,057,327.8	15.7	0
11	121	85	15	1,958,443.3	6,145.7	36,041.4	−0.005	806.3	—	3,681.4	1,958,345.6	11.0	−0.005
12	144	96	15	1,965,055.7	32,885.2	36,089.7	−1.420	878.1	—	3,725.0	1,937,152.3	17.9	−1.420

Notes. The time limit is 36,000 9seconds for CPLEX and 3,600 seconds for LocalSolver. Time1 is the time used for generating the best solution, whereas Time2 is the total runtime of CPLEX when the optimality of the solution is proved. We use the hub set found by Grid-VNS as the input of CPLEX and document the gap between the solution generated by CPLEX and time under the column of CPLEX2. For all instances that CPLEX generates optimal solutions, Grid-VNS coincides with CPLEX within 50 seconds. For some instances of these, Grid-VNS generates better solutions; thus, the Avg_Gap is less than zero. All gaps are expressed as a percentage, and the unit of time is seconds.

the best solution for every instance we tested within 10 times. Avg_Gap is less than 0 for some instances when n_b equals 11 and 12; that is, because for these cases, CPLEX terminates beforehand because of the time limit, whereas Grid-VNS still generates good solutions in a pretty short time. We report the average runtime for Grid-VNS to reach the Best_Obj as Time1 and visualize the comparison with an increasing number of grid cells with $p = 3$ and $p = 4$ in Figure A.1 (online appendix). When $n_b = 7$, Grid-VNS becomes one order of magnitude faster than CPLEX for these two instances. As the number of grid cells increases, the speed-up becomes more significant. We provide the decision of hubs to CPLEX and use it to find the optimal allocation (reported as CPLEX2). However, given that finding the optimal allocation is still an NP-hard problem, CPLEX does not generate the optimal allocation results in comparable time compared with the heuristics. CPLEX takes hundreds of seconds to find the optimal allocation for instances with $n_b = 12$, whereas it takes Grid-VNS less than 50 seconds to generate optimal solutions. The solutions found by LocalSolver are poor. Setting the time limit as 3,600 seconds, the heuristic is only able to generate

solutions for eight instances we tested, and only seven of them are optimal. LocalSolver is unable to generate feasible solutions for instances with $n_b > 11$ in less than 3,600 seconds.

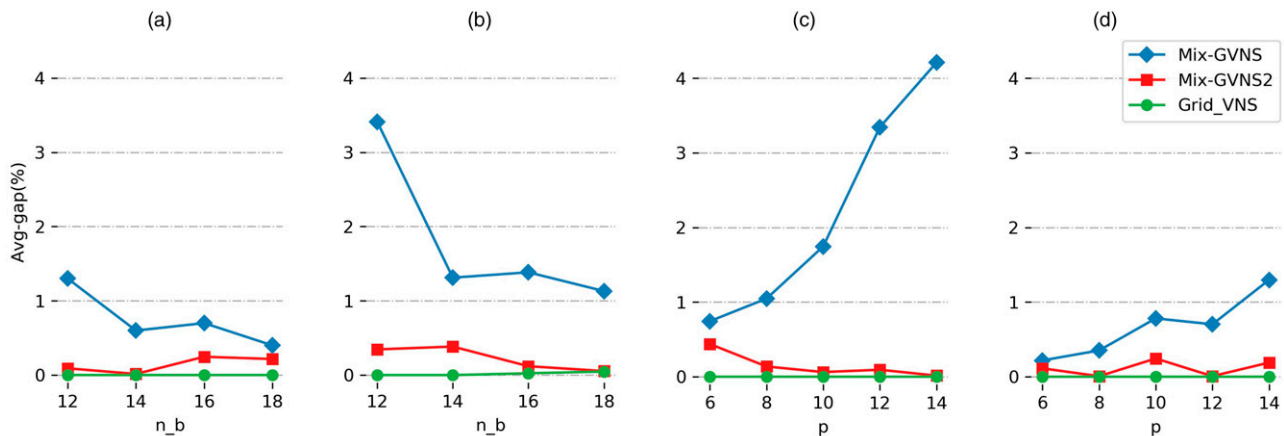
5.3.2. Comparison Between Other VNSs. To further evaluate the performance of Grid-VNS, we compare it with two other heuristics—Mix-GVNS and N-VNS. We first compare the performance on the solution quality. Every instance was run 10 times for GVNS, Grid-VNS, and N-VNS, respectively. In Table 5, we present the comparison of solutions generated by Mix-GVNS, Grid-VNS, and N-VNS, as well as the average CPU time for each heuristic to reach the best found solutions. Grid-VNS generates all of the best-known solutions each time for every instance when p is less than 15. For the two exceptions (instance with $n_b = 16, p = 15$ and $n_b = 18, p = 15$), the Avg_gap is smaller than 0.05%. The performance of N-VNS is close to Grid-VNS while using longer CPU time. This is because it does not explore the spatial information in the local searches. For all the instances with $p \geq 8$, Mix-GVNS is unable to generate the best solutions.

Table 5. Comparison Among Mix-GVNS, Grid-VNS, and N-VNS

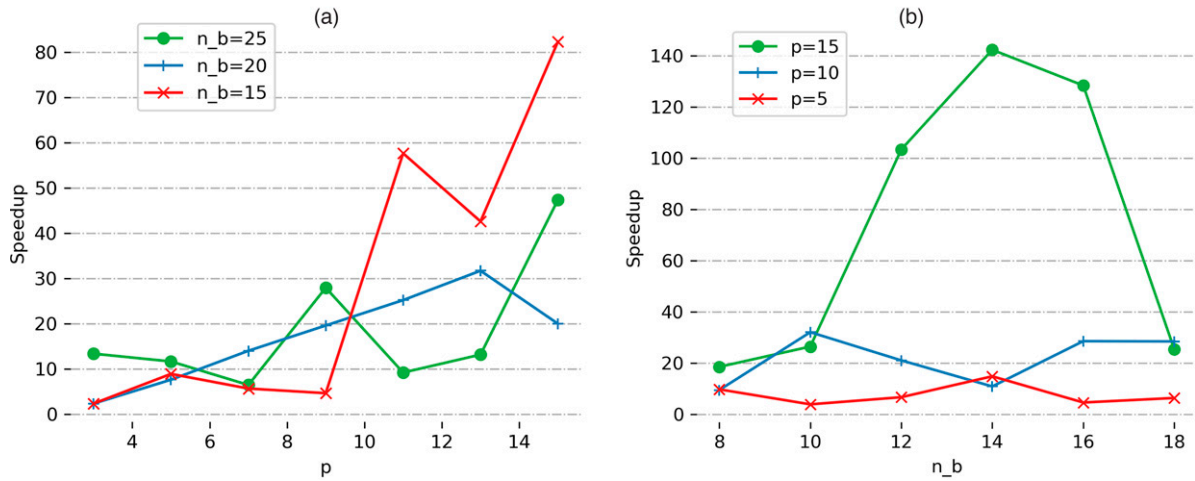
n_b	p	Best	Mix-GVNS				Grid-VNS				N-VNS			
			Avg_time	Min_gap	Avg_gap	No. of best	Avg_time	Min_gap	Avg_gap	No. of best	Avg_time	Min_gap	Avg_gap	No. of best
4	3	2,761,005.56	0.08	0	1.034	7	0.07	0	0	10	0.09	0	0	10
6		2,541,079.56	0.22	0	2.500	4	0.11	0	0	10	0.37	0	0	10
8		2,980,902.29	0.26	0	0	10	0.19	0	0	10	0.48	0	0	10
4	5	2,484,384.31	0.10	0	2.209	1	0.06	0	0	10	0.21	0	0	10
6		2,186,157.97	0.58	2.45	6.833	0	0.21	0	0	10	0.43	0	0	10
8		2,614,842.36	1.37	0	0.448	5	0.54	0	0	10	0.90	0	0	10
12		2,375,591.93	13.92	0	0.505	3	1.71	0	0	10	17.15	0	0	10
14		2,435,238.35	10.79	0	0.049	4	1.88	0	0	10	3.02	0	0	10
16		2,347,529.74	31.04	0	0	10	7.41	0	0	10	20.90	0	0	10
18		2,321,411.14	51.72	0	0.013	9	7.79	0	0	10	8.06	0	0	10
12	10	2,086,256.69	39.91	0.44	1.287	0	27.26	0	0	10	38.33	0	0	10
14		2,150,145.07	63.72	0.06	0.579	0	108.90	0	0	10	184.79	0	0	10
16		2,044,772.23	78.73	0.43	0.739	0	31.88	0	0	10	225.19	0	0	10
18		2,036,443.39	140.33	0.08	0.392	0	25.94	0	0	10	159.23	0	0.026	9
12	15	1,937,152.25	127.51	1.81	3.411	0	17.86	0	0	10	186.86	0	0	10
14		2,003,345.08	216.92	0.80	1.312	0	73.77	0	0	10	180.04	0	0	10
16		1,889,279.56	492.16	1.07	1.384	0	463.78	0	0.024	7	625.43	0	0.036	7
18		1,890,478.16	504.39	0.41	1.132	0	167.53	0	0.047	2	798.43	0	0.051	2
10	2	3,034,998.67	0.48	0	0	10	0.63	0	0	10	0.51	0	0	10
	4	2,603,594.61	4.04	0	0	10	0.74	0	0	10	0.98	0	0	10
	6	2,393,342.36	18.75	0	0.743	2	1.03	0	0	10	2.56	0	0	10
	8	2,294,004.33	35.06	0.44	1.046	0	5.84	0	0	10	19.40	0	0	10
	10	2,211,559.09	44.15	0.65	1.745	0	40.78	0	0	10	89.19	0	0	10
	12	2,144,198.41	26.68	1.22	3.344	0	8.30	0	0	10	79.20	0	0	10
	14	2,084,859.23	21.54	2.46	4.211	0	43.15	0	0	10	148.68	0	0	10
15	2	3,007,386.82	2.68	0	0	10	0.97	0	0	10	1.01	0	0	10
	4	2,521,589.07	17.34	0	0	10	1.84	0	0	10	2.23	0	0	10
	6	2,332,952.48	71.87	0	0.217	1	7.83	0	0	10	24.48	0	0	10
	8	2,239,999.77	82.30	0.05	0.351	0	61.25	0	0	10	68.44	0	0	10
	10	2,158,849.73	163.44	0.45	0.781	0	25.23	0	0	10	151.30	0	0	10
	12	2,094,035.64	331.51	0.09	0.701	0	74.78	0	0	10	172.21	0	0	10
	14	2,037,725.74	137.53	0.73	1.297	0	26.87	0	0	10	165.14	0	0	10

Notes. All instances are run 10 times for each method separately. Best refers to the best results obtained by these two methods for instances with $n_b > 10$ and optimal objective found by CPLEX for instances with $n_b \leq 10$. All gaps are expressed as a percentage. No. of best refers to the times that the best-known solution is reached within 10 runs for each algorithm. All gaps are expressed as a percentage, and the unit of time is seconds.

Figure 6. (Color online) Comparison of Average Deviation Between Mix-GVNS, Mix-GVNS2, and Grid-VNS



Notes. The deviation of Grid-VNS is the smallest among these three techniques for all of the instances. (a) $p = 10$. (b) $p = 15$. (c) $n_b = 10$. (d) $n_b = 15$.

Figure 7. (Color online) Visualization of the Speed-up Between the Runtime of Grid-VNS and Mix-GVNS, when Grid-VNS Generates Results not Worsen Than Mix-GVNS

Notes. Grid-VNS is faster than Mix-GVNS for all of the instances and the speed-up is more significant for large- p instances. (a) Fixed n_b , (b) Fixed p .

To further evaluate the solution quality between Grid-VNS and Mix-GVNS, we set the termination rule of Mix-GVNS as $20 \times n$ and use the average CPU time of Grid-VNS multiplied by 30 as the cut-off time (referred as Mix-GVNS2). For all of the instances shown in Table 5, the Avg_gap of Mix-GVNS is larger than 0. The complete results are presented in Table A.1 (online appendix), and selected results are visualized in Figure 6. For all instances, Mix-GVNS2 generated better results because of the increase of runtime. Only for the instances of $n_b = 4$, the average deviation of Mix-GVNS is zero, whereas for all other instances shown in Table A.1 (online Appendix), there still remains a large deviation compared with Grid-VNS.

Because the value of the objective function obtained by Grid-VNS is not worse than Mix-GVNS for every test we did, we compare the runtime when Grid-VNS finds solutions not worse than those generated by Mix-GVNS. The comparison between this runtime is visualized in Figure 7. We can observe that the runtime of Grid-VNS and Mix-GVNS is close when p is small. However, the speed-up becomes significant when p becomes larger, whereas there is no fixed pattern between speed-up and n_b . For all of the instances with $p = 15$, the speed-up is larger than 15. For instances with $p = 10$, the speed-up is larger than 9. The complete results are summarized in Table A.2 (online appendix).

5.3.3. Comparison Between Grid-VNS and IWA. In this section, we compare Grid-VNS with IWA, the most commonly applied iterative algorithm in the multifacility location problem. We compare the solution quality and runtime based on instances with $n_b \in$

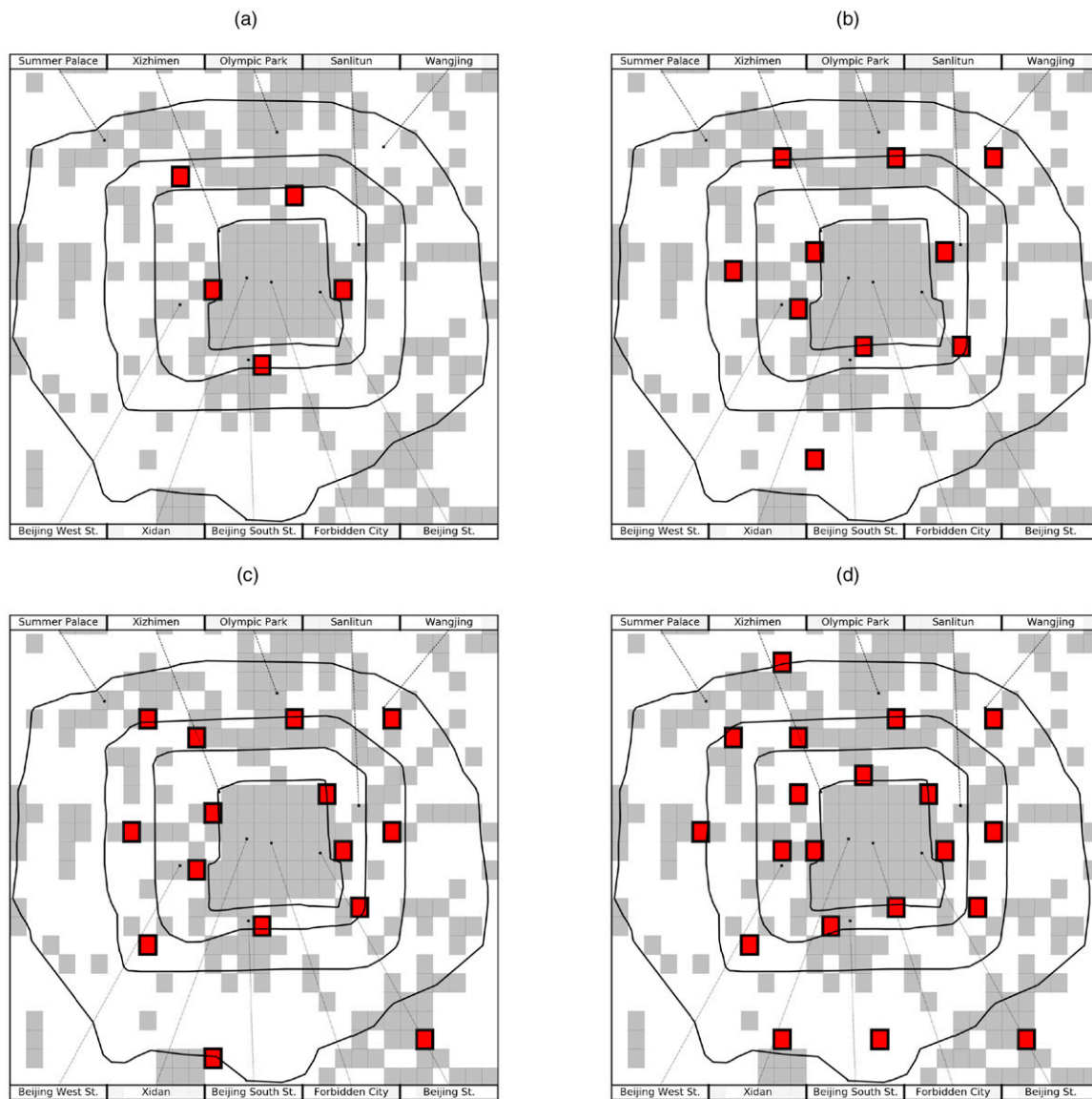
$\{13, 19, 25, 30, 35\}$ and $p \in \{2, 5, 10, 15, 20\}$. We randomly start IWA for 10 times and obtain the best solution found by IWA. The average deviation of IWA is calculated based on the solution generated by Grid-VNS. Although IWA terminates in a relatively short time for large-size instances, the deviation of IWA is very large. The runtime scales well as the p gets larger, but the solution quality worsens. As for Grid-VNS, it is able to generate results with a small deviation (with a maximum of 0.060% in all of the instances we tested). The complete results are summarized in Table A.3 (online appendix).

5.4. Optimal Vertiport Locations of Beijing

Applying our Grid-VNS, we obtain solutions for instances with different sizes and p . In Figure 8, the hub locations of instances with $n_b = 30$ and p in $\{5, 10, 15, 20\}$ are presented. As shown in Figure 8, when p is 5, all of the hubs are within the Forth Ring. As p increases, locations of hubs spread out and thus the area far away from the center can be served by a closer hub. We present the average distance and maximum distance for these four instances in Table 6. When there are only five hubs, the average distance from one grid cell to its hub is 8.77 km, whereas when there are 20 hubs, the average distance reduces to 4.21 km.

We present the visualization of solutions with the same p ($p = 10$) and different sizes in Figure 9. As shown in Figure 9, (a)–(c), for instances with the same p but different n_b , the assignments are close to each other, given that the demand of each grid cell is generated using the same travel record data set and the instances are intrinsically related. However, with larger n_b and smaller grid cells, the assignments have finer resolution. For

Figure 8. (Color online) Visualization of the Locations of Hubs for Instance with $n_b = 30$, and $p \in \{5, 10, 15, 20\}$.



Notes. The area filled with gray is the nonhubable area, and the grid cells with black edges are hubs. As p increases, locations of hubs spread out. (a) $p = 5$. (b) $p = 10$. (c) $p = 15$. (d) $p = 20$.

instances with the same n_b but different p , the topological structure of the clusters looks quite different. As shown in Figure 10, (a)–(d), the structure of clusters evolves from radial to annular shape as p increases.

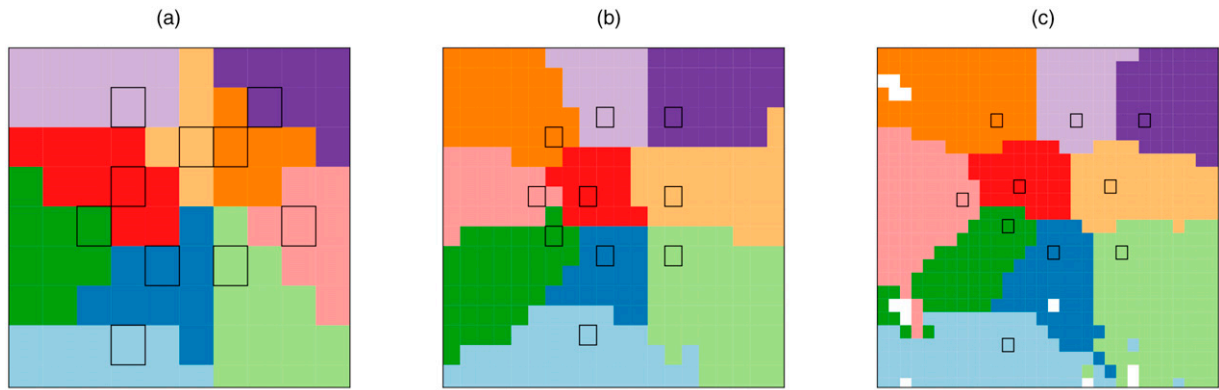
Table 6. Distance from Grid Cell to Its Hub for Instances with $n_b = 30$ and $p \in \{5, 10, 15, 20\}$

5		10		15		20	
max_ dis	avg_ dis	max_ dis	avg_ dis	max_ dis	avg_ dis	max_ dis	avg_ dis
22.57	8.77	19.84	6.12	15.47	4.67	14.14	4.21

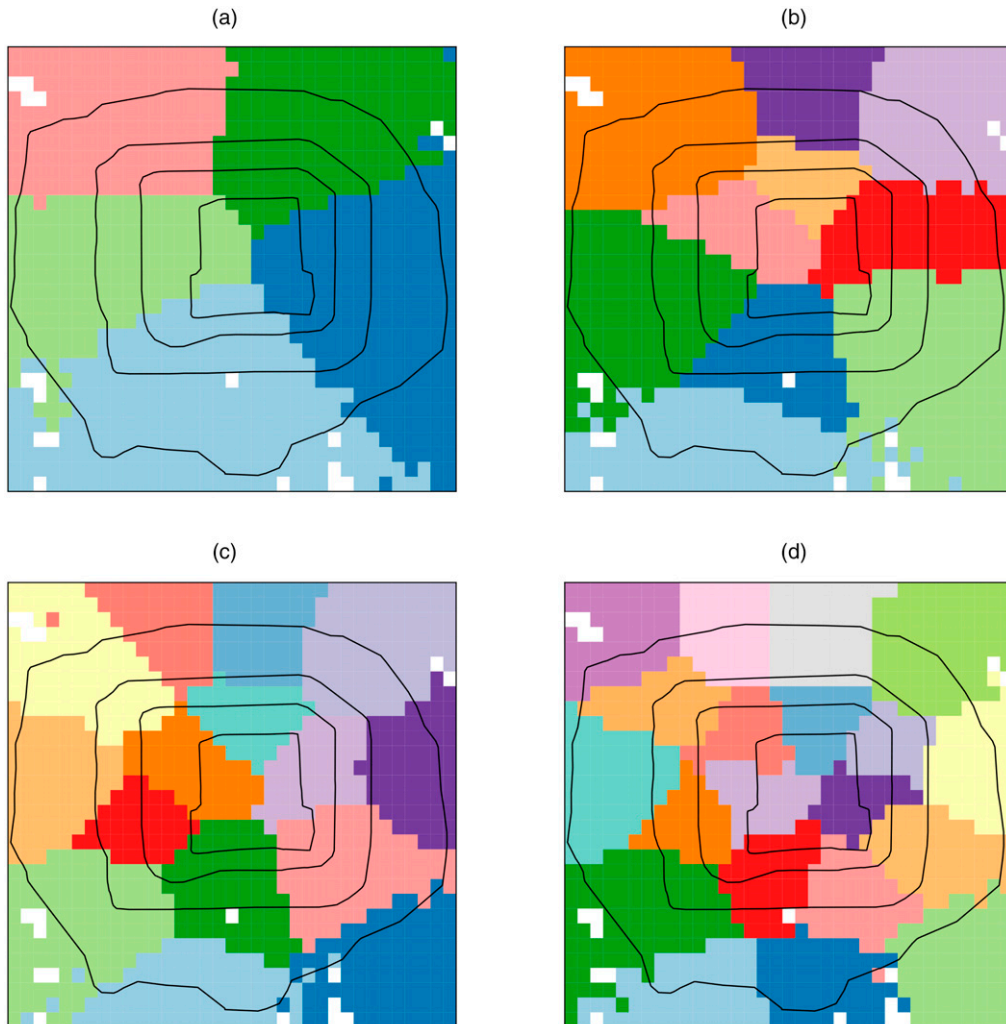
Notes. max_dis here refers to the maximum distance from a grid cell to its hub, and avg_dis refers to the average distance from every grid cell to its hub. All distances are in kilometers.

6. Conclusions

In this study, we focus on the locations of air taxi vertiports in a given metropolitan area with a goal to minimize the total travel cost. We allow the travel demand to be distributed over the whole metropolitan area, modeled as a grid, and accordingly propose our grid-based model for an uncapacitated single allocation p-hub median problem (USApHMP). Different from the previous studies on vertiport location problem, we consider the regions that vertiports are not allowed to be located in, for example, airspace restrictions and geographical constraints, and thus exclude certain grid cells from becoming hubs in our model.

Figure 9. (Color online) Visualization of Assignments with Different Sizes and a Fixed $p(p = 10)$ 

Notes. With larger n_b , the assignments have finer resolution. (a) $n_b = 10, p = 10$. (b) $n_b = 20, p = 10$. (c) $n_b = 30, p = 10$.

Figure 10. (Color online) Visualization of Assignments with a Fixed Size ($n_b = 35$) and Different p 

Notes. The structure of clusters evolves from radial to annular shape as p increases. (a) $n_b = 35, p = 5$. (b) $n_b = 35, p = 10$. (c) $n_b = 35, p = 15$. (d) $n_b = 35, p = 20$.

Although it is hard for exact techniques to solve large-size instances; for example, it takes commercial solver CPLEX more than 10 hours to generate results for instances with a grid division of 12×12 , we develop a grid-based variable neighborhood search heuristic (Grid-VNS) to solve this problem. Mixed variable neighborhood descent is used as the step of local searches that includes Alternate, Reassign, Explore, Substitute, and SwapK. Alternate and Reassign are previously studied in Mix-GVNS—one of the most efficient heuristics for USApHMP. However, we simplified the process of Alternate around $p/2$ times. Explore is a novel and efficient neighborhood search specifically designed for the grid-based model, whereas SwapK and Alternate can be applied in general USApHMP. These five local searches allow our heuristic to explore a larger neighborhood in comparable runtime compared with Mix-GVNS. Using Alternate, Reassign, Substitute, and SwapK, we construct a new VNS (N-VNS) for classic USApHMP.

Experimental results show that Grid-VNS outperforms Mix-GVNS in terms of runtime and the solution quality. Compared with CPLEX, Grid-VNS coincides with CPLEX for all instances with $n_b < 12$ we tested while using much shorter computation time. We also show that the results obtained by the continuous method (IWA) for large-size instances are poor, whereas Grid-VNS can generate results with a small deviation (with a maximum of 0.06%) for all instances.

This is an initial study on finding optimal vertiport locations when considering the nonhubable area. Our study can be further refined by considering other variants of hub location problems like the cost to build vertiports, incomplete network, different allocation strategies like multiple allocation and r-allocation, and capacity constraints. Other division schemes consisting of grid cells with different sizes can also be studied in future work.

References

- Abdinnour-Helm S (1998) A hybrid heuristic for the uncapacitated hub location problem. *Eur. J. Oper. Res.* 106(2-3):489–499.
- Alumur S, Kara BY (2008) Network hub location problems: The state of the art. *Eur. J. Oper. Res.* 190(1):1–21.
- Alumur SA, Kara BY, Karasan OE (2009) The design of single allocation incomplete hub networks. *Transportation Res. Part B Methodological* 43(10):936–951.
- Antcliff KR, Goodrich K, Moore M (2016) NASA Silicon Valley urban vtol air-taxi study. *On-demand mobility/emerging tech workshop*, vol. 7, Arlington.
- Calik H, Alumur SA, Kara BY, Karasan OE (2009) A tabu-search based heuristic for the hub covering problem over incomplete hub networks. *Comput. Oper. Res.* 36(12):3088–3096.
- Campbell JF (1993) Continuous and discrete demand hub location problems. *Transportation Res. Part B Methodological* 27(6):473–482.
- Campbell JF (1994) Integer programming formulations of discrete hub location problems. *Eur. J. Oper. Res.* 72(2):387–405.
- Chen JF (2007) A hybrid heuristic for the uncapacitated single allocation hub location problem. *Omega* 35(2):211–220.
- Contreras I, Díaz JA, Fernández E (2011) Branch and price for large-scale capacitated hub location problems with single assignment. *INFORMS J. Comput.* 23(1):41–55.
- Contreras I, Fernández E, Marín A (2010) The tree of hubs location problem. *Eur. J. Oper. Res.* 202(2):390–400.
- Corberán Á, Peiró J, Campos V, Glover F, Martí R (2016) Strategic oscillation for the capacitated hub location problem with modular links. *J. Heuristics* 22(2):221–244.
- Correia I, Nickel S, Saldanha-da Gama F (2018) A stochastic multi-period capacitated multiple allocation hub location problem: Formulation and inequalities. *Omega* 74:122–134.
- Cunha CB, Silva MR (2007) A genetic algorithm for the problem of configuring a hub-and-spoke network for a ltl trucking company in brazil. *Eur. J. Oper. Res.* 179(3):747–758.
- Dai W, Zhang J, Sun X, Wandelt S (2019) Hubbi: Iterative network design for incomplete hub location problems. *Comput. Oper. Res.* 104:394–414.
- Damgacioglu H, Dinler D, Ozdemirel NE, Iyigun C (2015) A genetic algorithm for the uncapacitated single allocation planar hub location problem. *Comput. Oper. Res.* 62:224–236.
- de Camargo R, Miranda G, Luna H (2008) Benders decomposition for the uncapacitated multiple allocation hub location problem. *Comput. Oper. Res.* 35(4):1047–1064.
- de Sá EM, Contreras I, Cordeau JF (2015) Exact and heuristic algorithms for the design of hub networks with multiple lines. *Eur. J. Oper. Res.* 246(1):186–198.
- Elhedhli S, Wu H (2010) A lagrangean heuristic for hub-and-spoke system design with capacity selection and congestion. *INFORMS J. Comput.* 22(2):282–296.
- Emir-Farinas H, Francis RL (2005) Demand point aggregation for planar covering location models. *Ann. Oper. Res.* 136:175–192.
- Ernst AT, Krishnamoorthy M (1999) Solution algorithms for the capacitated single allocation hub location problem. *Ann. Oper. Res.* 86(0):141–159.
- Ernst AT, Krishnamoorthy M (1996) Efficient algorithms for the uncapacitated single allocation p-hub median problem. *Location Sci.* 4(3):139–154.
- Eyster JW, White JA, Wierwille WW (1973) On solving multifacility location problems using a hyperboloid approximation procedure. *AIIE Trans.* 5(1):01–06.
- Farahani RZ, Hekmatfar M, Arabani AB, Nikbaksh E (2013) Hub location problems: A review of models, classification, solution techniques, and applications. *Comput. Industrial Engrg.* 64(4):1096–1109.
- Francis R, Lowe T, Tamir A, Emir-Farinas H (2004) A framework for demand point and solution space aggregation analysis for location models. *Eur. J. Oper. Res.* 159(3):574–585.
- Francis RL, Lowe TJ, Rayco MB, Tamir A (2009) Aggregation error for location models: Survey and analysis. *Ann. Oper. Res.* 167(1):171–208.
- Hansen P, Mladenović N (1997) Variable neighborhood search for the p-median. *Location Sci.* 5(4):207–226.
- He Y, Wu T, Zhang C, Liang Z (2015) An improved mip heuristic for the intermodal hub location problem. *Omega* 57:203–211.
- Hoff A, Peiró J, Corberán Á, Martí R (2017) Heuristics for the capacitated modular hub location problem. *Comput. Oper. Res.* 86:94–109.
- Holden J, Goel N (2016) *Uber Elevate: Fast-Forwarding to a Future of On-Demand Urban Air Transportation* (Uber Technologies).
- Holmes BJ, Parker R, Stanley D, McHugh P, Garrow L, Masson P, Olcott J (2017) *NASA strategic framework for on-demand air mobility*. Report for NASA Headquarters Aeronautics Research Mission Directorate.

- Holmes BJ (2016) A vision and opportunity for transformation of on-demand air mobility. *Proc. 16th AIAA Aviation Technology, Integration, and Operations Conf.*
- Holmes BJ, Durham MH, Tarry SE (2004) Small aircraft transportation system concept and technologies. *J. Aircraft* 41(1):26–35.
- Ilić A, Urošević D, Brimberg J, Mladenović N (2010) A general variable neighborhood search for solving the uncapacitated single allocation p-hub median problem. *Eur. J. Oper. Res.* 206(2):289–300.
- Kim HD, Perry AT, Ansell PJ (2018) A review of distributed electric propulsion concepts for air vehicle technology. *Proc. AIAA/IEEE Electric Aircraft Technologies Sympos.* (IEEE, New York), 1–21.
- Köksalan M, Soyulu B (2010) Bicriteria p-hub location problems and evolutionary algorithms. *INFORMS J. Comput.* 22(4):528–542.
- Kratka J, Milanović M, Stanimirović Z, Tošić D (2011) An evolutionary-based approach for solving a capacitated hub location problem. *Applied Soft Comput.* 11(2):1858–1866.
- Lim E, Hwang H (2019) The selection of vertiport location for on-demand mobility and its application to seoul metro area. *Internat. J. Aeronautical Space Sci.* 20(1):260–272.
- Lüer-Villagra A, Marianov V (2013) A competitive hub location and pricing problem. *Eur. J. Oper. Res.* 231(3):734–744.
- Martí R, Corberán A, Peiró J (2015) Scatter search for an uncapacitated p-hub median problem. *Comput. Oper. Res.* 58:53–66.
- Mladenović N, Urošević D, Hanafi S, Ilić A, et al (2012) A general variable neighborhood search for the one-commodity pickup-and-delivery travelling salesman problem. *Eur. J. Oper. Res.* 220(1):270–285.
- Mohammadi M, Julia P, Tavakkoli-Moghaddam R (2019) Reliable single-allocation hub location problem with disruptions. *Transportation Res. Part E Logistical Transportation Rev.* 123:90–120.
- O’Kelly ME (1986) The location of interacting hub facilities. *Transportation Sci.* 20:92–106.
- O’Kelly ME (1987) A quadratic integer program for the location of interacting hub facilities. *Eur. J. Oper. Res.* 32(3):393–404.
- O’Kelly ME, Campbell JF, de Camargo RS, de Miranda G Jr (2015) Multiple allocation hub location model with fixed arc costs. *Geographic Anal.* 47(1):73–96.
- Peiró J, Corberán A, Martí R (2014) Grasp for the uncapacitated r-allocation p-hub median problem. *Comput. Oper. Res.* 43:50–60.
- Polaczyk N, Trombino E, Wei P, Mitici M (2019) A review of current technology and research in urban on-demand air mobility applications. *Proc. Vertical Flight Society Autonomous VTOL Technical Meetings and Electronic VTOL Sympos.*
- Rath S, Chow JY (2019) Air taxi skyport location problem for airport access. Preprint, submitted April 1, <https://arxiv.org/abs/1904.01497>.
- Robinson JN, Sokollek MDR, Justin CY, Mavris DN (2018) Development of a methodology for parametric analysis of stol airport geodensity. *Proc. Aviation Technology, Integration, and Operations Conf.*
- Rodríguez-Martín I, Salazar-González JJ, Yaman H (2014) A branch-and-cut algorithm for the hub location and routing problem. *Comput. Oper. Res.* 50:161–174.
- Schippl J, Decker M, Fleischer T (2013) Personal air vehicles as a new option for commuting in europe: Vision or illusion? *Proc. 41st European Transport Conf.*
- Serper EZ, Alumur SA (2016) The design of capacitated intermodal hub networks with different vehicle types. *Transportation Res. Part B Methodological* 86:51–65.
- Stumpf E, Kreimeier M, Strathoff P, Lueckhof JM, Schroeder KU (2018) Small aircraft concept for regional on-demand air mobility. *Proc. 31st ICAS Congress.*
- Sun X, Wandelt S, Stumpf E (2018) Competitiveness of on-demand air taxis regarding door-to-door travel time: A race through europe. *Transportation Res. Part E Logistical Transportation Rev.* 119:1–18.
- Sun X, Dai W, Zhang Y, Wandelt S (2017) Finding-hub median locations: An empirical study on problems and solution techniques. *J. Adv. Transportation.*
- Todosijević R, Urošević D, Mladenović N, Hanafi S (2017) A general variable neighborhood search for solving the uncapacitated r-allocation p-hub median problem. *Optim. Lett.* 11(6): 1109–1121.
- Yaman H (2011) Allocation strategies in hub networks. *Eur. J. Oper. Res.* 211(3):442–451.

Copyright 2022, by INFORMS, all rights reserved. Copyright of Journal on Computing is the property of INFORMS: Institute for Operations Research and its content may not be copied or emailed to multiple sites or posted to a listserv without the copyright holder's express written permission. However, users may print, download, or email articles for individual use.